

Table of Contents

3D Video Conferencing

Introduction.....	page 1
System Requirements.....	page 2
Development Tools.....	page 2
Biography.....	page 3
Chapter 1 (Opening Screens).....	page 4
Opening Screen.....	page 4
Opening Screen Functionality.....	page 5
Chapter 2 (File Screen).....	page 6
File Screen Functionality.....	page 6
File Screen Exit.....	page 7
Chapter 3 (Settings).....	page 8
Settings Functionality.....	page 8
Functionality (cont.).....	page 9
Chapter 4 (Ports Tab).....	page 10
Ports Tab Functionality.....	page 10
Functionality (cont.).....	page 11
Ports Changed	page 12
Chapter 5 (Cameras Tab).....	page 13
Cameras Tab Functionality.....	page 13
Functionality (cont.).....	page 14
Device Changed.....	page 15
Quality Changed.....	page 16
Chapter 6 (Audio Tab).....	page 17
Audio Tab Functionality.....	page 17
Audio Output Changed.....	page 18
Audio Input Changed.....	page 19
Chapter 7 (Main Screen).....	page 20
Main Screen Functionality	page 20
Main Screen Functionality (cont.).....	page 21
Chapter 8 (Expanded Screen).....	page 22
Expanded Screen Functionality.....	page 22
Preview Option.....	page 23
3D Video Option.....	page 24
Voice Option.....	page 25
Webcam Close.....	page 26

Introduction

3D Video Conferencing

3D Video Conferencing is a program designed to utilize two DirectShow compatible web cameras to create a more “immersive” and 3D experience for the party participating in a video conference. The frames from each of the web cameras rapidly interleaving are outputted to the user’s screen. Using eDimensional’s LCD (Liquid Crystal Display) shutter glasses, the interleaving frames create the illusion of 3D.

The primary benefits for the 3D Video Conferencing program are it’s relatively cheap price (compared to other video conferencing programs) and ease of acquisition (two web cameras, LCD shutter glasses and driver software provided by eDimensional).

Applications for 3D Video Conferencing include:

- Family and friends can communication with others.
- Teachers can lecture to a remote group of students.
- Soldiers in the armed forces can use it to communication with family members and friends.
- Security personnel can gain a more “realistic” perspective of intruders.

System Requirements

Windows PC 95/98/ME/2000/XP

Intel Pentium® III 550 MHz (Intel Pentium 4 1.4 GHz recommended) or higher.

CRT or LCD external monitor with VGA connection (no laptop screens)

- Minimum 70Hz refresh rate suggested

256 MB of RAM or higher.

Display adapter capable of 16-bit color depth or higher, 2 MB or higher video memory.

- nVidia recommended

Development Tools

Microsoft® Visual Studio 2008

Microsoft® .C# Express Edition 2008

Microsoft® .NET Framework 2.0

DirectShowNet Library

Un4seen Bass.NET 2.4 API

Biography



Philip Andal is a Charleston Southern University graduate in Computer Science. He has a B.S. from Charleston Southern University and also attended the College of Charleston. He welcomes feedback about his project by email at

philip.andal@gmail.com



Jeremy Tibayan is a Charleston Southern University graduate in Computer Science. He has a B.S. from Charleston Southern University. He welcomes feedback about his project by email at

jtibayan1@gmail.com

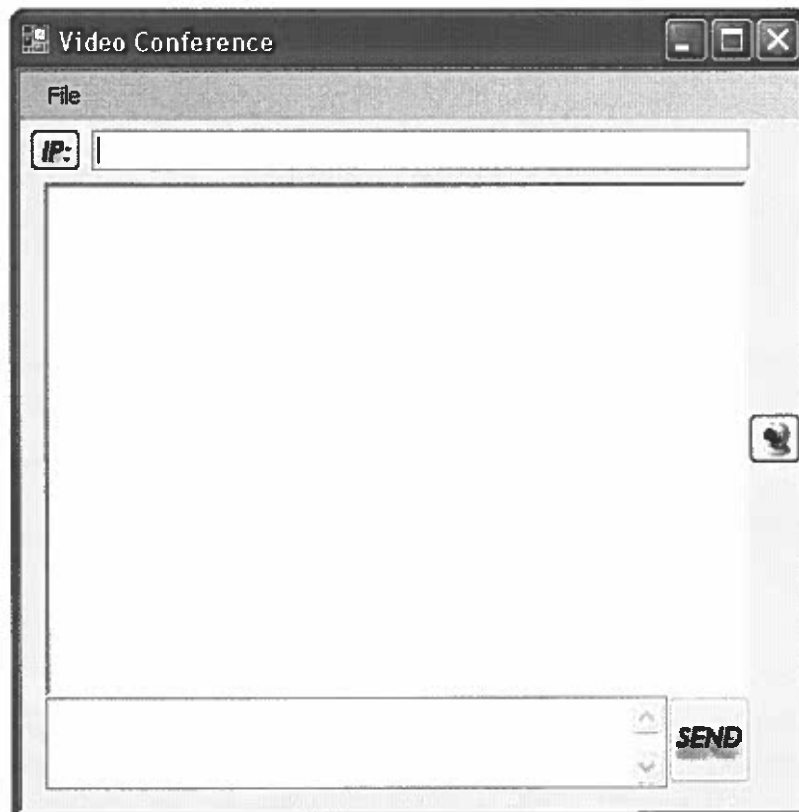
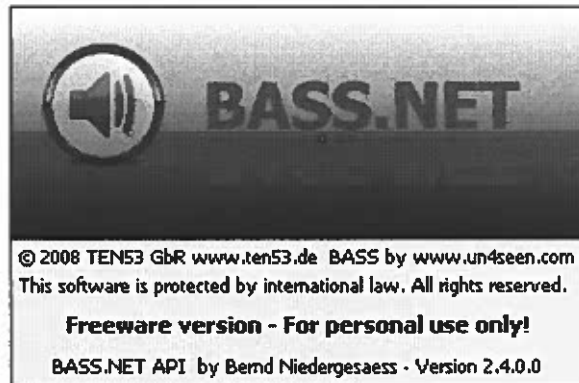


Dr. Lin is a professor of Computer Science at Charleston Southern University. He has a PH. D from the University of Florida. He advises Philip Andal and Jeremy Tibayan on the 3D video conferencing project. He welcomes feedback about the project by

email at ylin@csuniv.edu.

Chapter 1

OPENING SCREENS



FUNCTIONALITY

Screen	Description
Bass.NET	Informs the use of the Un4seen Bass.NET API

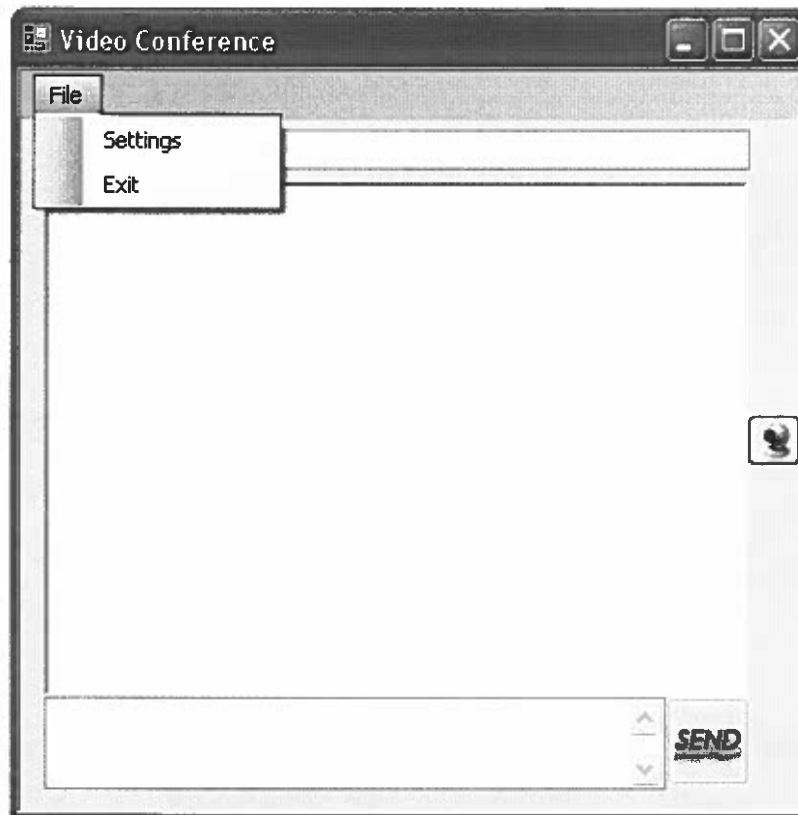
Button	Description
File	Opens the ability for Settings and Exit
IP	Shows the IP Address of the machine
Send	Sends a string as a text message to the end user
Webcam	Expands the size of the form to reveal other options

Field	Description
IP	Shows the IP Address of the machine communicating to
Text Message	Shows the message typed or being typed to be sent

Chapter 2

3D Video Conferencing

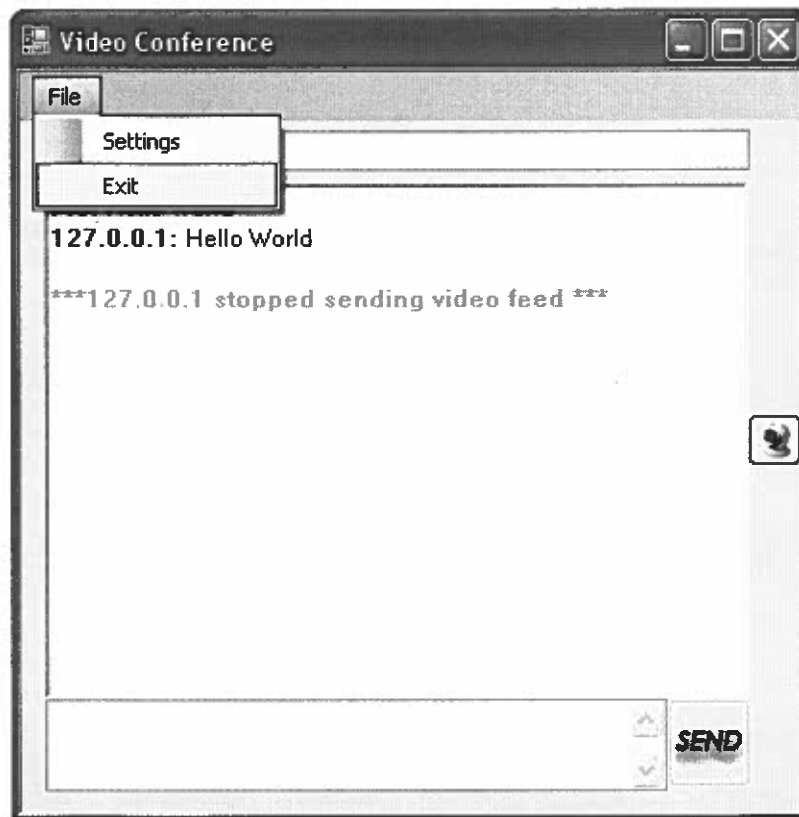
FILE SCREEN



FUNCTIONALITY

Field	Description
Settings	Opens another window where settings of the program can be changed
Exit	Exits out of the program

FILE EXIT



FUNCTIONALITY

Field	Description
Settings	Opens another window where settings of the program can be changed
Exit	Exits out of the program and keeps the settings (if changed prior to exit of program)

Chapter 3

3D Video Conferencing

SETTINGS

The screenshot shows a Windows-style dialog box titled "Settings". It has three tabs: "Ports", "Cameras", and "Audio". The "Ports" tab is selected. Inside the dialog, there are three text input fields with labels to their left: "Text Message Port:" with the value "8070", "Video Port:" with the value "8080", and "Audio Port:" with the value "8060". At the bottom of the dialog, there is a note that says "NOTE: Must restart program for settings to take effect!". Below the note are two buttons: "Cancel" and "Apply".

FUNCTIONALITY

Tabs	Description
Ports	Textboxes that show each of the port numbers for: Text Message, Video and Audio
Camera	Drop boxes that allow the user to select which web cameras they intend to use
Audio	Drop boxes that allow the user to select which audio device to use for recording audio and which device for outputting of audio

Button	Description
Cancel	Cancels any changes made to any port number or any other setting that has been altered
Apply	Applies the changes (if any) made to port numbers, web camera devices and audio devices

Chapter 4

PORTS TAB

Settings

Ports Cameras Audio

Text Message Port: 8070

Video Port: 8080

Audio Port: 8060

NOTE: Must restart program for settings to take effect!

Cancel Apply

FUNCTIONALITY

Field	Description
Text Message Port	Shows the port where text messaging (incoming and outgoing) is passing. The port number is defaulted to 8070
Video Port	Shows the port where video (incoming and outgoing) is passing. The port number is defaulted to 8080
Audio Port	Shows the port where audio (incoming and outgoing) is passing. The port number is defaulted to 8060

Button	Description
Cancel	Cancels any changes made to any port number or any other setting that has been altered
Apply	Applies the changes (if any) made to port numbers, web camera devices and audio devices

PORTS CHANGED

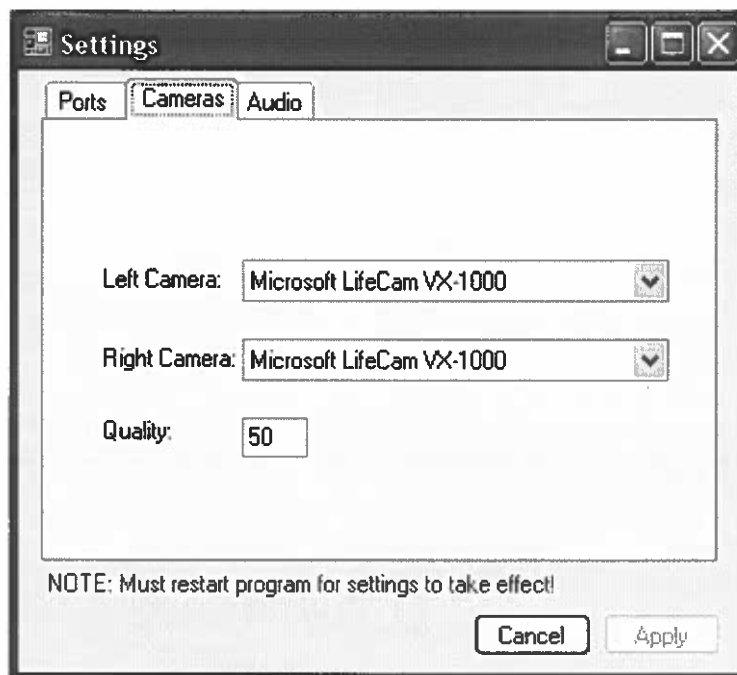
The screenshot shows a Windows-style dialog box titled "Settings". It has three tabs: "Ports", "Cameras", and "Audio". The "Ports" tab is selected. Inside the dialog, there are three text input fields: "Text Message Port:" with the value "8070", "Video Port:" with the value "8080", and "Audio Port:" with the value "8090". At the bottom of the dialog, there is a note that says "NOTE: Must restart program for settings to take effect!". Below the note are two buttons: "Cancel" and "Apply".

FUNCTIONALITY

Button	Description
Cancel	Cancel any changes made to the port numbers and exits out of "settings"
Apply	Applies any and all changes made to port numbers and exits of "settings"

Chapter 5

CAMERAS TAB

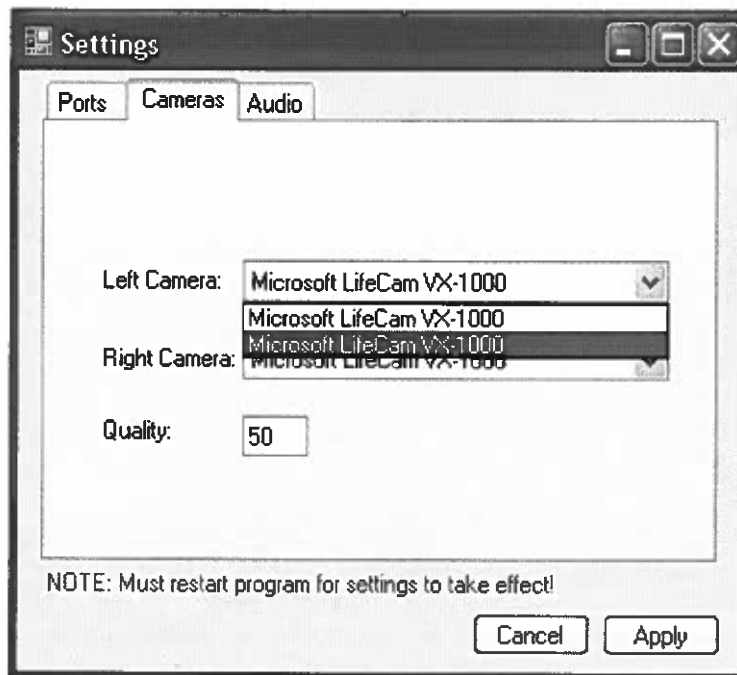


FUNCTIONALITY

Field	Description
Left Camera	Drop box that reveals other web cameras or other equivalent devices to use as the "Left Camera"
Right Camera	Drop box that reveals other web cameras or other equivalent devices to use as the "Right Camera"
Quality	Sets the quality of the video feed being sent. It is defaulted to 50. Range: 1-100

Button	Description
Cancel	Cancels any changes made to any port number or any other setting that has been altered
Apply	Applies the changes (if any) made to port numbers, web camera devices and audio devices

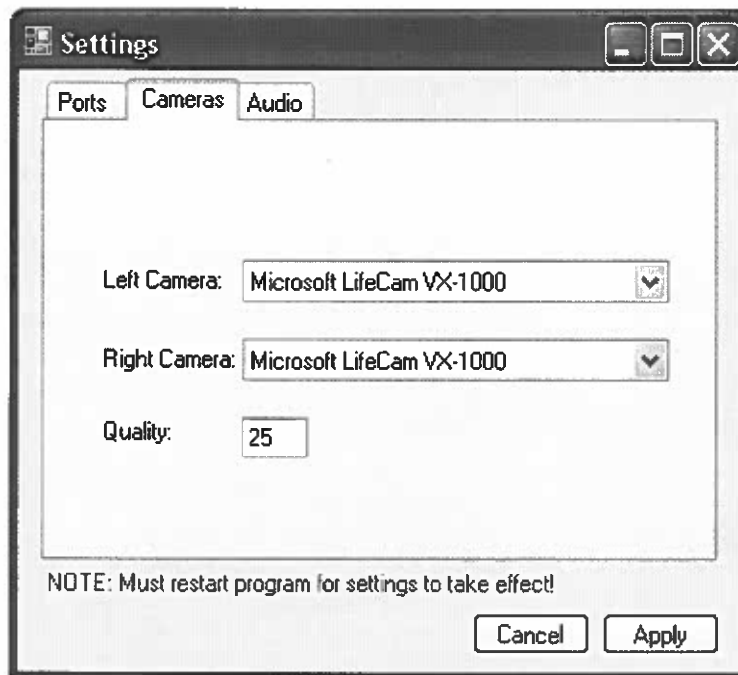
DEVICE CHANGED



FUNCTIONALITY

Button	Description
Cancel	Cancel any changes made to the selected devices (left and right) and exits out of "settings"
Apply	Applies any and all changes made to the devices and exits out of "settings"

QUALITY CHANGED

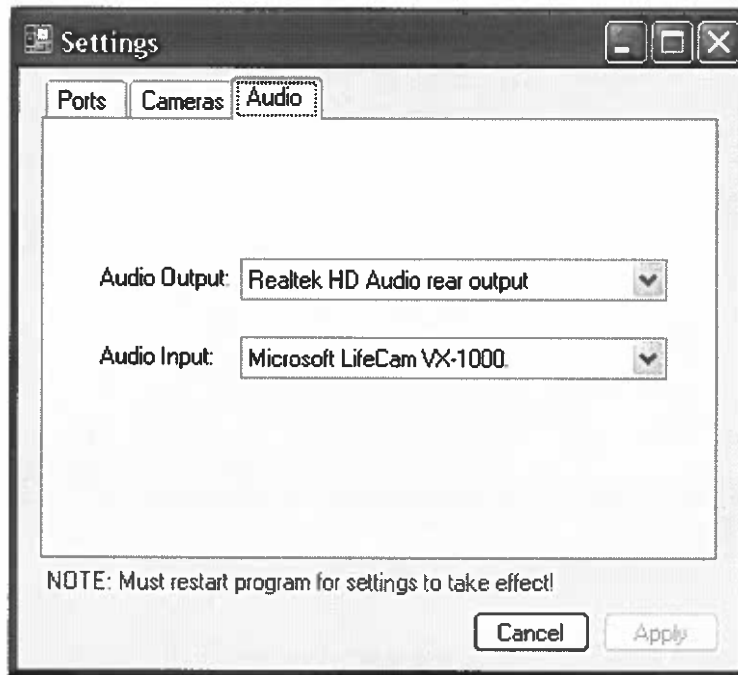


FUNCTIONALITY

Button	Description
Cancel	Cancel any changes made to the video quality and exits out of "settings"
Apply	Applies changes made to the video quality and exits out of "settings"

Chapter 6

AUDIO TAB

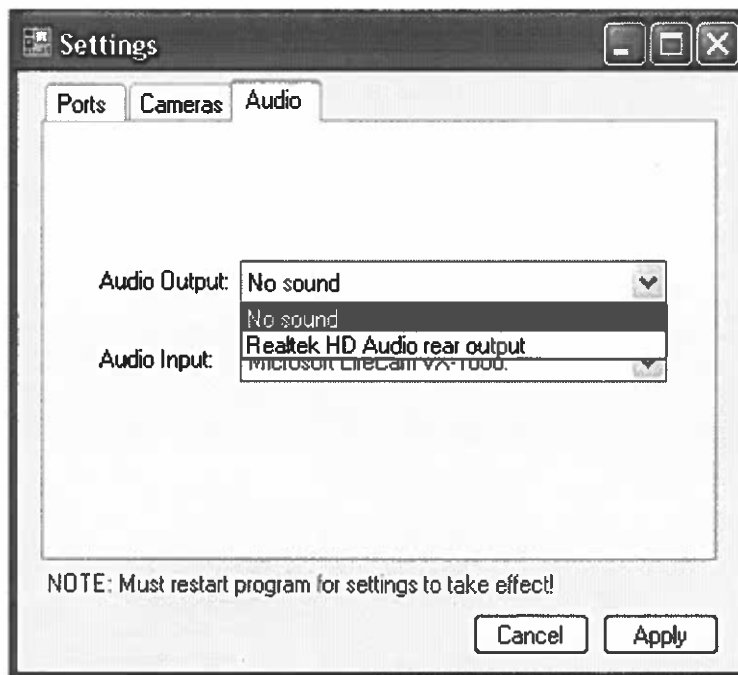


FUNCTIONALITY

Field	Description
Audio Output	Drop box that reveals other audio output or other equivalent devices to use as the "Audio Output"
Audio Input	Drop box that reveals other audio devices or other equivalent devices to use as the "Audio Input"

Button	Description
Cancel	Cancels any changes made to any port number or any other setting that has been altered
Apply	Applies the changes (if any) made to port numbers, web camera devices and audio devices

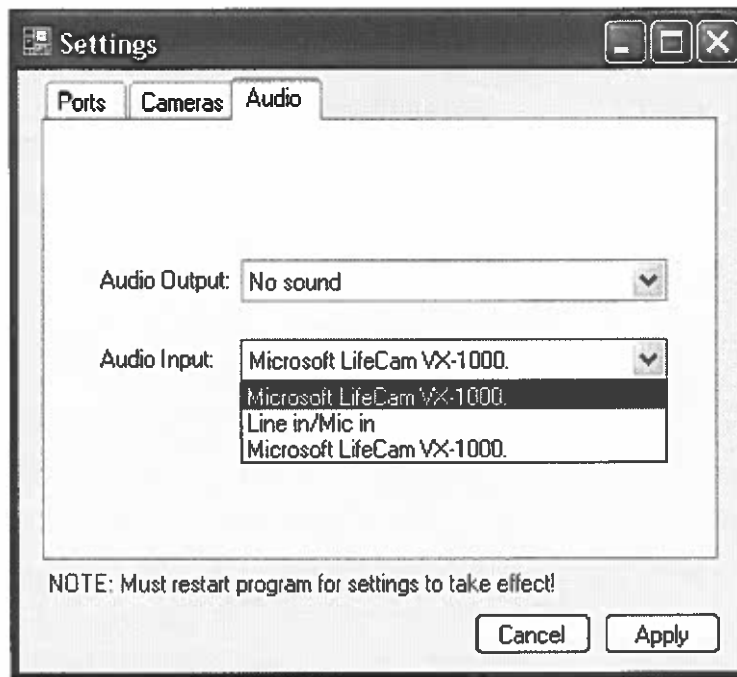
AUDIO OUTPUT CHANGED



FUNCTIONALITY

Button	Description
Cancel	Cancel any changes made to the audio output and exits out of "settings"
Apply	Applies changes made to the audio output and exits out of "settings"

AUDIO INPUT CHANGED



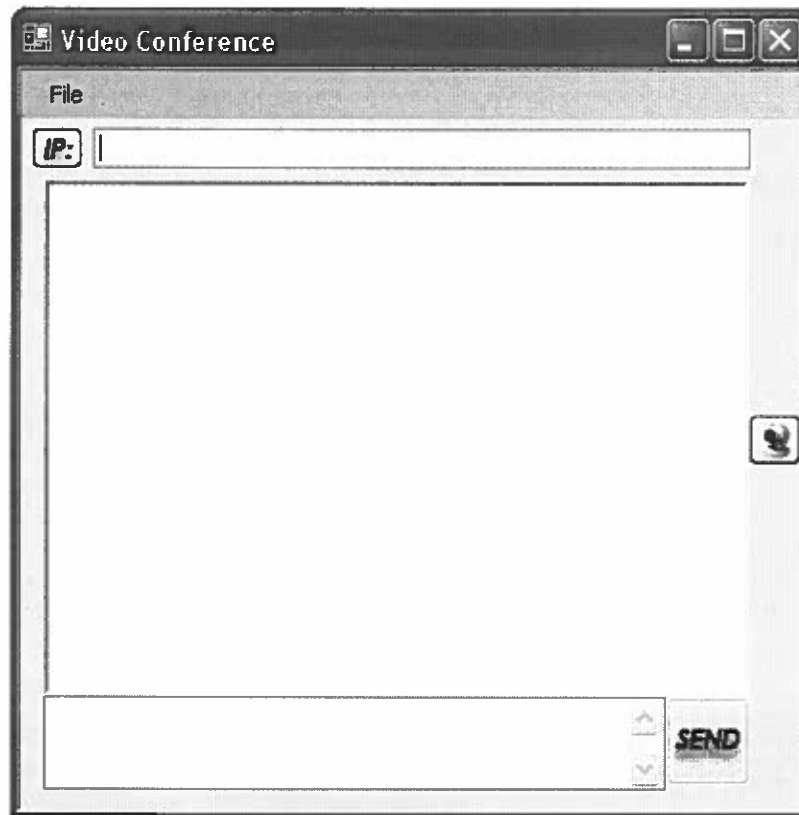
FUNCTIONALITY

Button	Description
Cancel	Cancel any changes made to the audio input and exits out of "settings"
Apply	Applies changes made to the audio input and exits out of "settings"

Chapter 7

3D Video Conferencing

MAIN SCREEN



FUNCTIONALITY

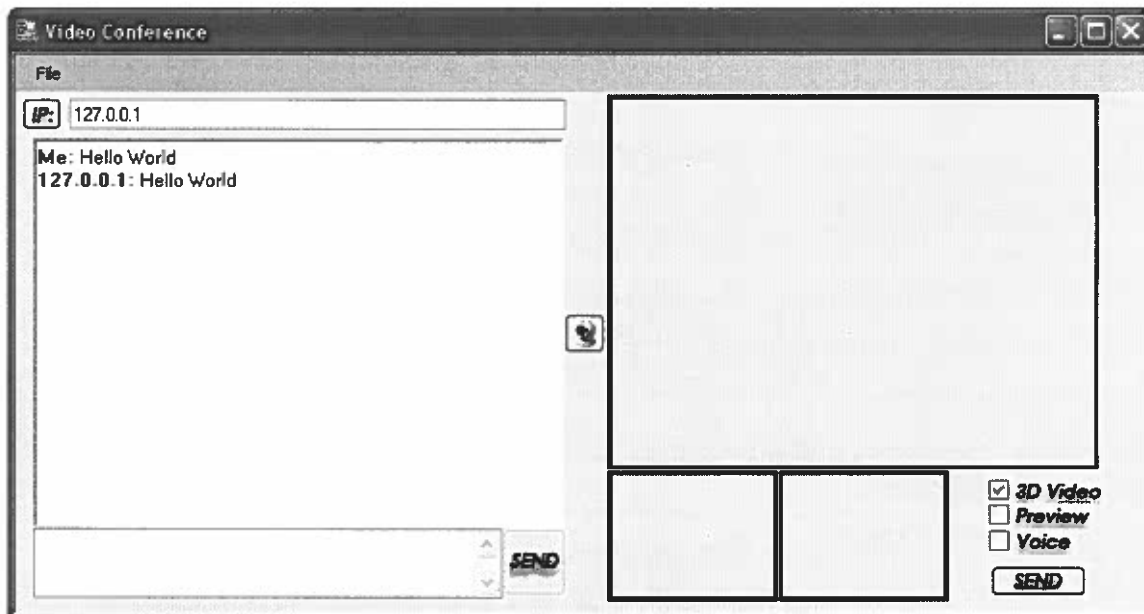
Field	Description
IP	Shows the IP Address of the machine communicating to
Text Message	Shows the message typed or being typed to be sent

Button	Description
IP	Shows the IP Address of the machine
Send	Sends the message typed
Webcam	Expands the form size for web camera and audio options

Chapter 8

3D Video Conferencing

EXPANDED SCREEN

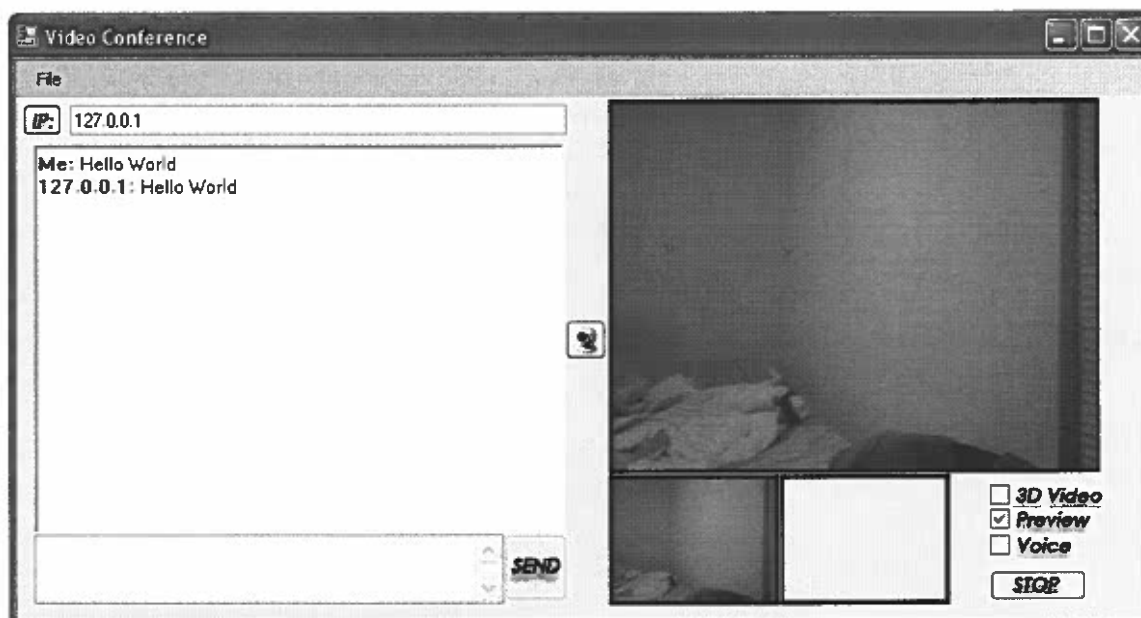


FUNCTIONALITY

Checkbox	Description
3D Video	When the box is checked, 3D Video is enabled. When the box is unchecked, the feed from the first (default) web camera is sent
Preview	When the box is checked, a preview is shown (smaller boxes) for each web camera under the larger box
Voice	When the box is checked, voice is enabled

Button	Description
Send	Depending on the check boxes, what ever is checked is sent to the end user

PREVIEW OPTION

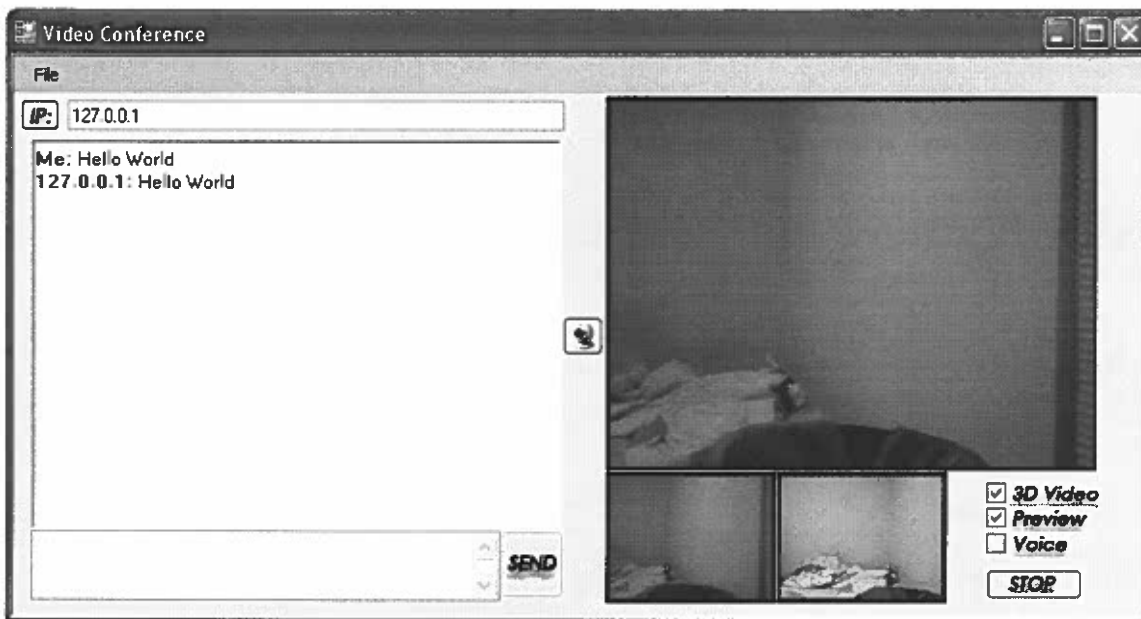


FUNCTIONALITY

Checkbox	Description
3D Video	The feed from the first (default) web camera is sent
Preview	A preview is shown (smaller boxes) for each web camera under the larger box
Voice	When the box is checked, voice is enabled

Button	Description
Stop	Stops what ever options are checked from the check boxes.
Webcam	Minimizes the form size to it's original form
Send	Sends what ever is typed or being typed to the end user

3D VIDEO OPTION

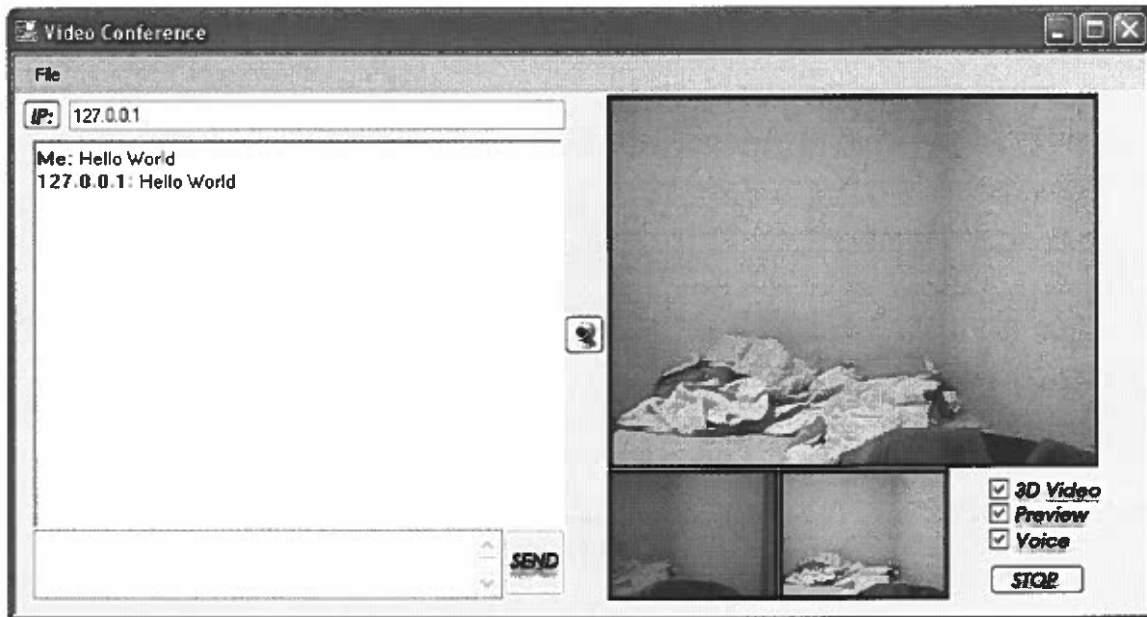


FUNCTIONALITY

Checkbox	Description
3D Video	3D Video is enabled. Interleaved frames from each device selected for the “Left” and “Right” cameras are outputted
Preview	A preview is shown (smaller boxes) for each web camera under the 3D Video
Voice	When the box is checked, voice is enabled

Button	Description
Stop	Stops what ever options are checked from the check boxes.
Webcam	Minimizes the form size to it’s original form
Send	Sends what ever is typed or being typed to the end user

VOICE OPTION

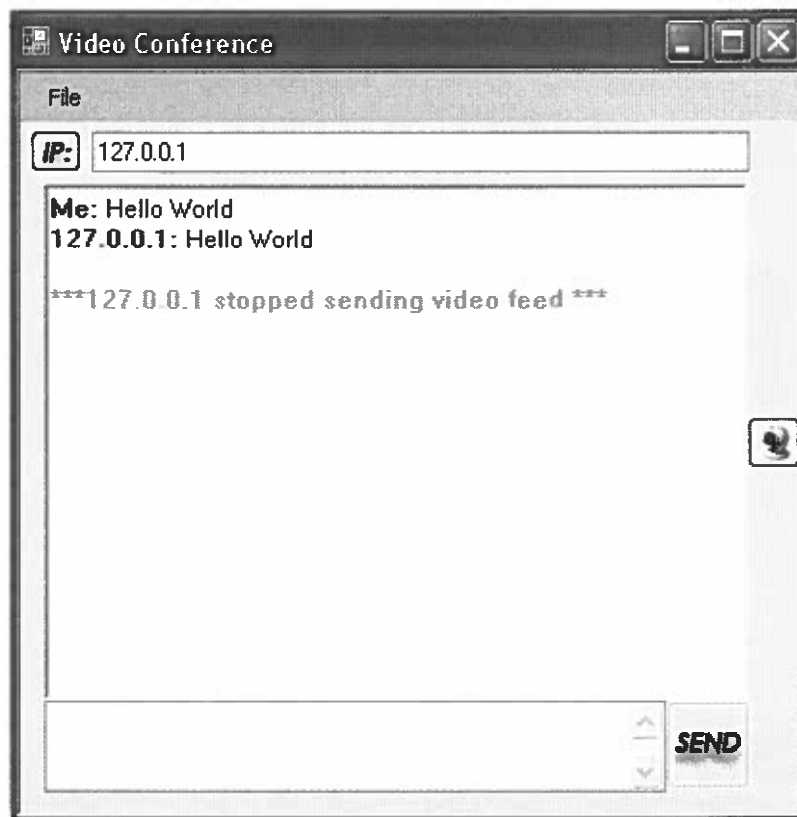


FUNCTIONALITY

Checkbox	Description
3D Video	3D Video is enabled. Interleaved frames from each device selected for the “Left” and “Right” cameras are outputted
Preview	A preview is shown (smaller boxes) for each web camera under the larger box
Voice	Voice is enabled. What ever recording device is selected (default if user does not choose prior to sending), audio is recorded and sent

Button	Description
Stop	Stops what ever options are checked from the check boxes.
Webcam	Minimizes the form size to it’s original form
Send	Sends what ever is typed or being typed to the end user

WEBCAM CLOSE



FUNCTIONALITY

Button	Description
IP	Stops the IP Address of the machine
Webcam	Expands the form size to reveal more options: 3D Video, Preview and Voice
Send	Sends what ever is typed or being typed to the end user

Table of Contents

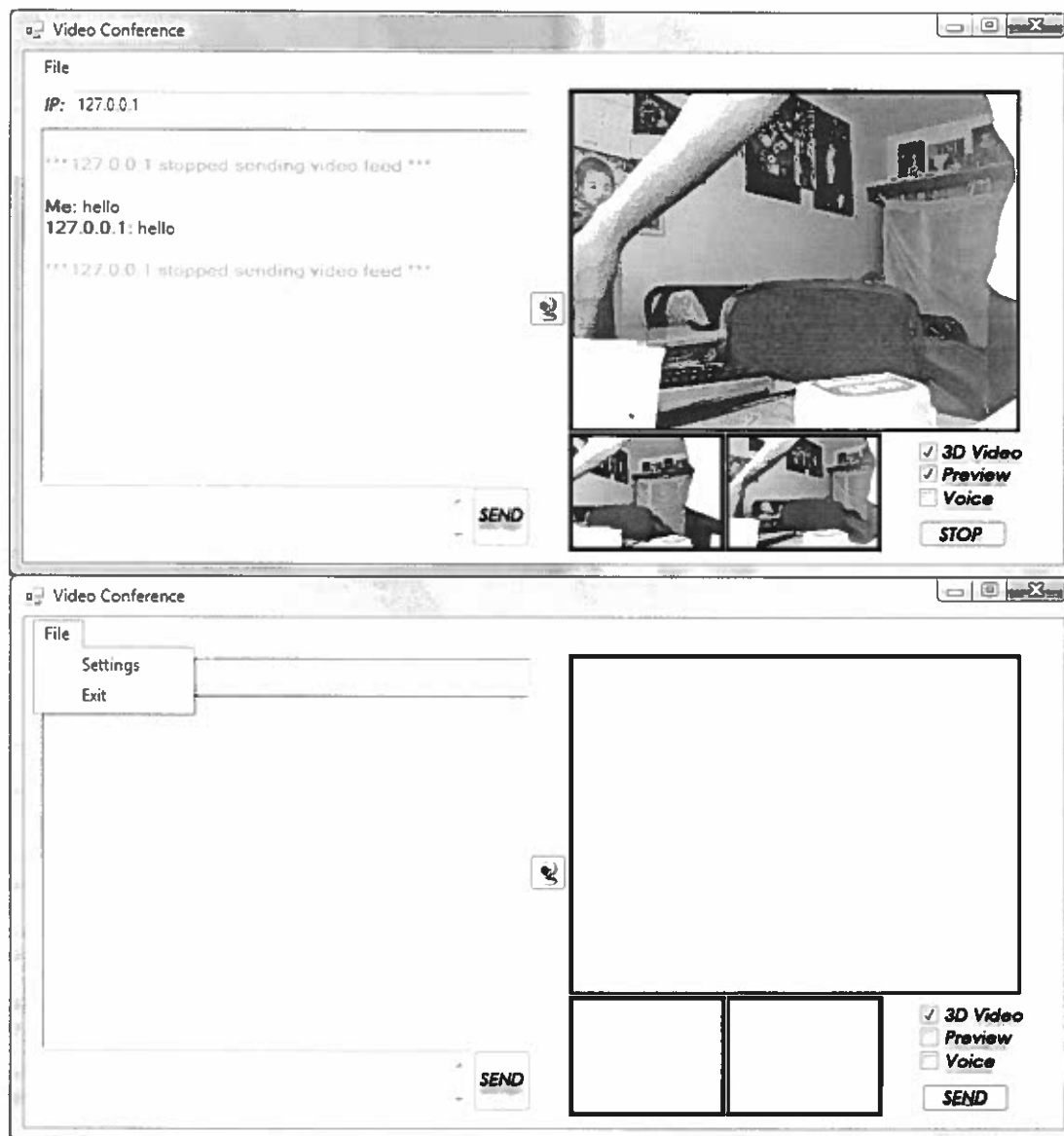
3D Video Conference

Form1.cs	01
SettingsForm.cs	06
VideoClass.cs	09
Capture.cs	11
VoiceClass.cs	14

Form1.cs

3D Video Conference

Form1.cs is the main form of the program. Everything starts from Form1.cs. The interface contains four buttons, three picture boxes, three text boxes, three check boxes, and a file menu with two entries.



This form allows the user to start a text, video, and/or voice conference with another user over a local network. The IP box shows the address of the remote user and it will automatically be filled with an IP address from which received data is coming from. The next text box, the larger one, contains any received and sent text messages. It distinguishes between the remote user texts and the local user texts. The third text box is where the user types in a text message to be sent. The button with the camera icon, when pressed, will slide open the form to reveal the video communication options. The three check boxes are for the optional 3D video, preview, and voice communication. The two smaller picture boxes contains an optional preview of what a user is sending over the network to the other peer involved in the conference. The 320x240 picture box contains the video feed that is being received. The last two buttons, one being hidden, the 'Send' and 'Stop' buttons which enable and disable the sending of a video feed. From the file menu the user can open up the settings form or exit the program.

Member Variables and Procedure

Variable	Description
VideoClass vid	vid is a VideoClass object where all the video methods are contained.
VoiceClass voice	voice is a VoiceClass object where all the audio methods are contained
const int FRAMERATE = 30	Used when initializing cameras within a VideoClass object
const int HANGUP = -1	The value sent to the remote user when the sending of a video feed is stopped
const int CALL = 1	Used for the call setup (call set up does not work so this goes unused)
const int INTERVAL = 33	The interval in which a Windows.Forms.Timer is to grab images from the capture devices
const int FREQ	The frequency of the audio (optional voice)
const string LOCALHOST = "127.0.0.1"	If no IP is specified by user, remote IP address will default to LOCALHOST
Queue<Image> img_q	A queue for all the images coming in
MemoryStream in_ms	Received packets will be converted to this memory stream and will then be converted to an image

Socket udpSocket	All outgoing text data will be sent through this socket
Thread msgRecvThread	This thread takes in all incoming text messages
Thread recvVoiceThread	This thread receives all incoming audio packets
Thread recvThread	This thread receives all incoming video frames
string ipAddress	This is the variable for the address of the remote user
Windows.Forms.Timer myTimer	Timer used to grab and send images from capture device
Windows.Forms.Timer slideOutTimer and slideInTimer	These timers are used to control the sliding in and out of the form
Timers.Timer dispImgTimer	This is the timer that handles the frames store in the img_q
Windows.Forms.Form settingForm	Variable for the settings form
Font stdFont, idFont	Font options for the text message box
bool exitFlag, camsOn, voiceON, preview	Various flags for the different options of video communication
long IMG_QUALITY	This is set to the user specified option in the settings form
Image imagw	Variable for the optional preview
Control mainForm	A control to the main form (form1) so that the program can change dimensions of the form
int sizecount	Int used for sliding in and out
PictureBox[] preBox	Array of 2 picture boxes for left and right video previews
PictureBox[] pBox	Variable for the main picture box in which incoming video feed will be shown

Procedure	Parameters	Description
Form1()	none	Everything in this function happens before anything else on load. It sets the mainForm control and sets Double Buffer (for less flicker) to true
Form1_Load()	- object sender - EventArgs e	This is Form1's load event. Many of the member variables are initialized here. All threads (receiving threads) are started here as well.
dispRemIP()	string ip	This function displays the IP address of the remote user that is sending data
hideShowButton()	Button b	Hides or shows a button you pass to it
dipsPic()	MemoryStream ms	Displays the image frame passed to it in the receiving picture box
dispImgEvent()	object o	The function for the

	• System.Timers.ElapsedEventArgs e	System.Timers.Timer which looks at the img_q for frames and displays them
receivingThread()	• none	This is the function called by recvThread. It receives any incoming video data
button1_Click()	• object sender • EventArgs e	This is the stop buttons click event. It will stop the sending of video and audio if audio is enabled
btn_send_Click()	• object sender • EventArgs e	The send button click event which starts the sending of video and optionally audio
SendVideo()	• Object myObject • EventArgs myEventArgs	This is one of the functions called when the send button click event occurs. It sends the video frames at the user specified image quality. This is also where the preview is enabled if the user wants it.
StartCams()	• None	Starts the capture graphs of the capture devices specified in the settings form
CheckBox changed Events	• object sender • EventArgs e	All the check box changed events will set/change the associated flags
textMsgBox_TextChanged()	• object sender • EventArgs e	When the input text message box contains user text, the send button will be enabled
textMsgBox_keyDown() textMsgBox_PreviewKeyDown()	• object sender • KeyEventArgs e or PreviewKeyDownEventArgs e	Both of these functions looks for the user to press the 'Enter' key in the input text message box and if it is pressed it will call the text message send function
btnText_Click()	• object sender • EventArgs e	The click event for the text message send button. Displays and sends the message
dispMsg()	• string msg • bool id • Color c	Displays the msg string in the received text message box. If it's the ID of the user it sets its font properties accordingly with Color c.
textReceiving()	• none	Function called by the text receiving thread. Receives all incoming text messages and displays them in the received message box by calling dispMsg()
textSend()	• string str	Sends the text message str to the remote user
StartVoice()	• none	Starts the transmission of voice from the selected audio input

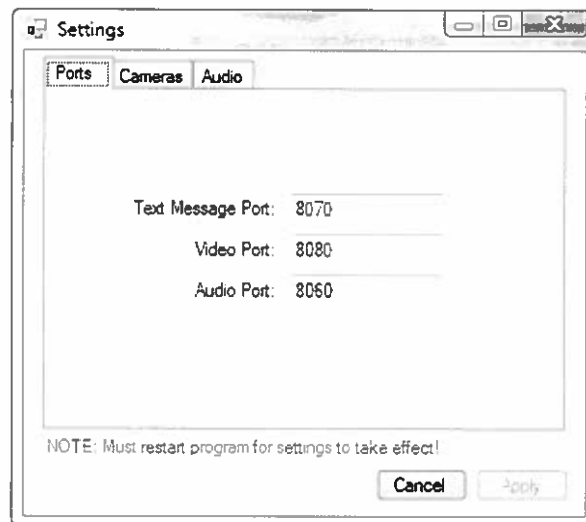
		device
recvVoice()	• none	The function called by the receiving voice thread. Receives all audio packets and sends them to the selected audio output device
btnSlide_Click()	• object sender • EventArgs e	Click event for the slide button that slides the form in or out
beginSlide()	• None	Slides the form
slideOut() slideIn()	• Object myObject • EventArgs myEventArgs	Slides the form out Slides the form in
exitToolStripMenuItem_Click()	• object sender • EventArgs e	The click event for the exit file menu item that exits the program
settingsToolStripMenuItem_Click()	• object sender • EventArgs e	The click event for the settings file menu item that opens up the settings form
exitProc()	• None	The function called when user exits the program
Form1_Closing()	• object sender • ComponentModel.CancelEventArgs e	Calls exitProc()

SettingsForm.cs

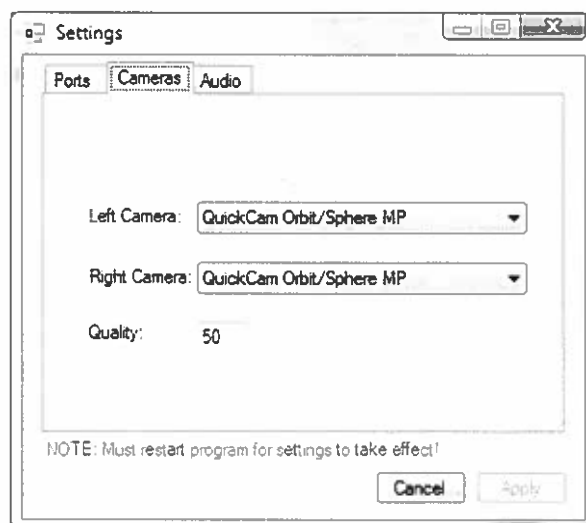
3D Video Conference

SettingsForm.cs is the form containing all the user changeable settings.

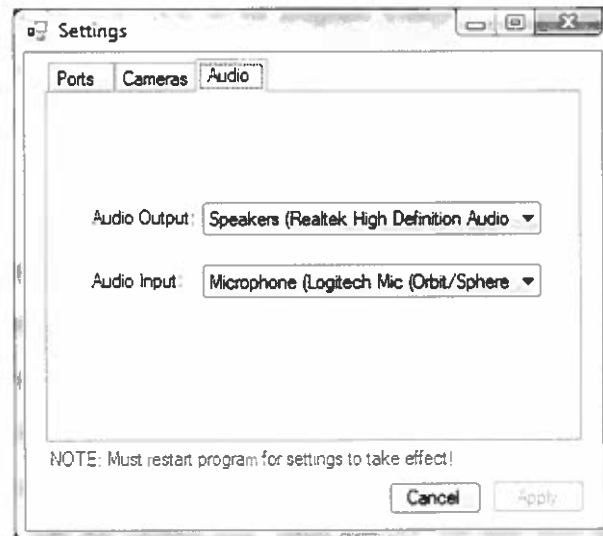
There are three tabs in the form:



The first tab, the ports tab, contains three text boxes for the three network ports for text, video, and audio communication.



The second tab, the cameras tab, contains two drop down boxes and a text box. The two drop down boxes list all the available capture devices and the user picks one for the left and one for the right. The selected devices must be unique to each other. The text box in this tab contains an integer value from 1-100 for the image quality.



The third tab, the audio tab, contains two drop down boxes. The first drop down box is for the audio output device while and second drop down box is for the audio input device.

There are two buttons, a cancel and apply button. The apply button is enabled when settings are changed and if pressed will save all the changes and exit the settings form. The cancel button undoes any changes made and closes the settings form.

When settings are applied, a program restart should be performed to ensure all newly adjusted settings take effect.

Member Variables and Procedures

Variable	Description
bool changed	The flag that specifies whether any setting has been changed
ArrayList deviceList	An array used to store all the capture devices found connected to the computer
const string portError	The message notifying the user of the port range allowed for any of the three ports
struct DeviceStruct	A structure for numbering and naming each capture device. Also contains a function for override what the ToString() function would return for this structure

Procedure	Parameters	Description
SettingsForm()	none	Loads the drop boxes and applies settings stored in the Properties.Settings.settings
LoadDropBoxes()	none	Loads the drop boxes in the cameras and audio tabs with capture and audio devices
getDevices()	none	Returns an ArrayList of DeviceStructs when called
ApplySettings()	none	Looks at the Properties.Settings.settings table and fills in all the text boxes and drop down boxes of all the tabs
SaveSettings()	none	This saves all the settings to Properties.Settings.settings if any of the settings has changed and are applied. Makes sure user does not select any erroneous changes.
applyBtn_Click()	- object sender - EventArgs e	The Apply button click event that calls SaveSettings() to save all the settings and closes the settings form
SettingsChanged()	- object sender - EventArgs e	The event triggered whenever any of the controls in the form are changed. When triggered, the Apply button will be enabled
cancelBtn_Click()	- object sender - EventArgs e	The cancel button click event that closes the settings form without saving any changes
SettingsForm_Load()	- object sender - EventArgs e	The load event of the settings form. Disables the apply button immediately on load.

VideoClass.cs

3D Video Conference

VideoClass.cs is the class used to set up two capture devices using the Capture.cs class. It initializes the capture devices and contains functions for sending video feeds along with options of preview, image quality, frame rate, and 3D video.

Member Variables and Procedures

Variable	Description
const int LEFT = 0, RIGHT = 1	Simple constants as indexes for the two item array of capture devices. Left device being at the 0 index and the right device at the 1 index.
int selDev	Will be set to the current device that the frame is being captured from.
Capture[] cam	An array of two Capture objects
int leftDev, rightDev, frameRate	The ints used in initializing the array of Capture class objects.
bool vid3D	Determines whether or not to send interleaving frames or a feed from just one device
bool helper3D	Boolean used to switch back and forth the device used to capture the current frame
long vidQuality	Used to determine the quality of the video
Socket sock	The socket used to send outgoing video feed
IPEndPoint IEP	Remote destination where video feed will be sent to
Image currImg	Current image variable for the optional preview

Procedure	Parameters	Description
VideoClass()	- int left - int right - int iFrameRate	Constructor for the video class. left is the device index for the left capture device, right is the device index for the right capture device, and iFrameRate at which the capture graph of each device will be running at
setLeft()	int device	Sets the left capture device
setRight()	int device	Sets the right capture device
video3D()	bool confSetting	Set whether 3D video is enabled or not
setQuality()	long quality	Sets the video quality
Init()	none	Initializes the capture devices
Start()	none	Calls Init() and starts both left and right capture devices
Pause()	int dev	Pauses the capture graph of device dev
Stop()	none	Stops the capture graph of all currently

		running devices
getPreview()	• out int handle	Returns the current frame for preview and also sends out which device the frame is coming from
Send()	• string ip • int port	Grabs frame from the current device and sends them over the network. Can send from both devices or from a single device depending on settings selected by user
getImage()	• int device	Returns a memory stream of what the current frame is of device dev
~VideoClass()	• none	Class destructor. Closes the socket and call Stop()
GetEncoderParams()	• long quality	Returns encoding parameters for the video quality
GetEncoderInfo()	• string mimeType	Returns the encoder info of mimeType

Capture.cs

3D Video Conference

Capture.cs is a DirectShowNet class and is used by VideoClass to set up the capture graphs of each camera using DirectShow. Contains functions for setting up all the DirectShow interfaces for capturing frames from the capture devices. Previously contained a memory leak in GetBitMap() function. Must free memory as shown in line 130.

Member Variables and Procedures

Variable	Description
IFilterGraph2 m_FilterGraph	The DirectShow interface for the filter graph
IMediaConrol m_mediaCtrl	DirectShow interface for controlling the capture graph of capture devices
ManualResetEvent m_PictureReady	Notifies a waiting thread that a captured frame is ready
bool m_bGotOne	Information of the graph status
IntPtr m_handle	Pointer used to copy image data into
int m_videoWidth, m_videoHeight, m_stride	Various properties of the capture graphs. m_videoWidth is the width of the frames, m_videoHeight is the height of the frames, and m_stride is the number of bytes allocated for one scanline of each frame
int m_Dropped	Count of dropped frames

Procedure	Parameters	Description
Capture()	- int iDeviceNum - int iFrameRate	Capture class constructor. Calls _Capture() to initialize device index and frame rate of the Capture class.
Dispose()	none	Closes all DirectShow interfaces and releases all used resources. Called by Capture class destructor
~Capture()	none	Capture class destructor. Calls Dispose()
Width	none	Gets the video frame width
Height	none	Gets the video frame height
Stride	none	Gets the video frame stride
GetBitmap()	none	Returns a frame captured from the capture graph. Allocates memory for m_handle. Waits for a frame to be grabbed from the capture graph. If a frame is grabbed, an image will be

		created from data pointed to by m_handle and the pixels will be flipped as when an image is grabbed it is grabbed upside down. Free the memory after a successful grab and save or after an exception is thrown and return the frame captured. Exception is thrown when a frame isn't returned within five seconds.
Start()	• none	Starts the capture graph of the initialized capture device
Pause()	• none	Pauses the capture graph. A running capture graph takes up a lot of resources, pausing it whenever it's not needed will save some of those resources
_Capture()	• int iDeviceNum • int iFrameRate • int iWidth • int iHeight	Called by the class constructor to initialize all device properties. Enumerates all the devices connected to the system and sets up the capture graph of the device at device index iDeviceNum. The graph is set up with iFrameRate, iWidth, and iHeight. If any parameter is set to 0, the graph will take on default properties.
SetupGraph()	• DsDevice dev • int iFrameRate • int iWidth • int iHeight	This function builds the capture graph for frame grabbing. It will capture frames from DirectShow device dev. The graph will run at iFrameRate at height iHeight and width iWidth. If any parameter is 0, default values will be used. Setting up the capture graph requires getting ICaptureGraphBuilder2 and SampleGrabber interfaces. Starts building the graph using m_FilterGraph. Adds the video device to the graph, configures the sample grabber, and adds the frame grabber to the graph. The stream is then rendered and resources used to build the graph are released.
SaveSizeInfo()	• ISampleGrabber sampGrabber	Saves properties (height, width, and stride) of the video stream
ConfigureSampleGrabber()	• ISampleGrabber sampGrabber	Configures the sample grabber with media types.
SetConfigParms()	• ICaptureGraphBuilder2 capGraph • IBaseFilter capFilter • int iFrameRate	Sets the frame rate and the video size of the video stream

	• int iWidth • int iHeight	
CloseInterfaces()	• none	Shut down the capture graph
ISampleGrabberCB.BufferCB()	• double SampleTime • IntPtr pBuffer • int BufferLen	Thread safe buffer callback. Copies frame data from a location in memory to m_handle.
isRunning()	• none	Returns a bool of whether or not the capture graph is running

Additional Information

#region API

```
[DllImport("Kernel32.dll", EntryPoint="RtlMoveMemory")]
private static extern void CopyMemory(IntPtr Destination, IntPtr
    Source, int Length);
```

#endregion

This region is used to import a method from the Kernel32.dll. CopyMemory() copies a specified length data from one memory location to another.

VoiceClass.cs

3D Video Conference

VoiceClass.cs is the class used to set up the audio input and output devices. Along with initializing the devices, it is also responsible for sending the audio data through the network to the remote user very similar to how the VideoClass works. This class makes use of the Bass.NET API which is free for personal use. Future development should not use this API and should use DirectSound. It should also be a part of the video class. The methods used in this class were strictly for testing purposes as audio was not a priority when creating this project.

Member Variables and Procedures

Variable	Description
private int recDev, outDev, frequency	These ints are for the use of initializing the audio devices. recDev is the recording device or audio input, outDev is the output device, and frequency is the frequency of the audio
int recStream, playStream	These are the recording and playback streams. Both are initially set to -1 to determine whether they were initialized by the program yet
RECORDPROC recProc	A delegate of the Bass.NET API
byte[] localbuffer	The local buffer storage for recorded audio to be sent over the network
Socket sock	Socket used to send all outgoing audio data
IPEndPoint IEP	Remote endpoint destination to send audio data to
bool sending	Set to true when audio is enabled

Procedure	Parameters	Description
VoiceClass()	- int rDev - int oDev - int freq	Constructor for the VoiceClass. rDev is the index of the recording device (microphone), oDev is the index of the audio output device (speakers), and freq is the frequency of audio.
Init()	none	Initiates the recording device and the audio output device with specified frequency. It will stop any audio samples currently being played through Bass.NET and will start the recording and playback streams.
Record()	none	Initializes the delegate function with the recorder() function. Will create a handle for the recording stream and start it. It also sets playStream to a Bass.NET push stream.
recorder()	int handle	Function that copies data from the recording stream in memory to localbuffer. Every time there is data in the buffer

	• IntPtr buffer • int length • IntPtr user	for the specified audio period in the Record() function, it will send that data over the network if the sending flag is set to true.
PlayBuffer()	• byte[] buffer	Takes buffer, a byte array, and plays back the audio data stored in it.
Play()	• none	Starts or resumes the playback of an audio sample
Send()	• string ipAdd • int p	Sets the sending boolean to true to notify the RECORDPROC delegate to send audio data to ipAdd through port p
~VoiceClass()	• none	VoiceClass destructor. Will stop the playback and recording of any streams. Frees all resources used.

Email

3D Video Conference

We wanted to gain more information as to how to correctly view and sync our 3D Video Conferencing program with eDimensional's 3D LCD (Liquid Crystal Display) shutter glasses. After obtaining an email address related to the eDimensional Company, we decided to ask them questions. The following email is what was sent.

From: Philip Andal [mailto:peandal@csustudent.net]
Sent: Wednesday, November 28, 2007 9:56 PM
To: sales@edimensional.com
Subject: Question

Hello eDimensional,

My name is Philip Andal and I am currently working on a senior project with another student here at Charleston Southern University in South Carolina. Our advisor for our project is Dr. Yu-Ju Lin, ylin@csuniv.edu. I know this is the sales email address but I could not locate an appropriate address to send this email. If it is possible, could you forward this message to whom it may concern.

The reason I'm writing you is because we are using your 3D glasses for our project. Our final project is going to, hopefully, display a 3D video stream onto the computer screen and with the help of your glasses the user will be able to see this 3D feed.

My question to you: is it possible to sync 2D images with the glasses to create a 3D image?

**Thank you so much for your time,
Philip Andal**

Response Email

3D Video Conference

Within a day, we received a prompt and helpful email from Michael Epstein, Customer Sales, from eDimensional. It lead us to more tips and ideas. It also lead us to the right direction for our concept. The following email is his reply to our message.

----- Original Message -----

From: "Michael Epstein" michael@eDimensional.com

To: peandal@csustudent.net

Sent: Thu 29/11/07 8:55 AM

Subject: Fwd: RE: Question

Hello,

Thank you for contacting eDimensional. To view images in 3D, you will have to have some application that converts them in real time into a format such as stereoscopic interlaced that will synch with our glasses. The glasses do not automatically convert everything on your monitor into 3D.

I'm sure you'll be thrilled with the E-D glasses and accessories and please let me know if you have any other questions. We look forward to receiving your order.

Regards,

Michael
Customer Sales
eDimensional, Inc.
www.eDimensional.com
Tel: 561-833-2199
Toll Free Sales Hotline 1-866-TO BUY E-D

New G-Pad Pro Motion Sensing Rumble Gamepad for the PC or PS!! The Latest eDimensional Innovation: http://www.edimensional.com/product_info.php?cPath=24&products_id=118 *

Learning

3D Video Conference

eDimensional has two different versions of their 3D glasses; one being wired and the other being wireless. The eDimensional LCD shutter glasses that were provided by the school are the wireless version. Signals are sent from the IR (infrared) transmitter that connects to the eDimensional Dongle. This Dongle is connected to the user's VGA (Video Graphics Array) port on their computer.

eDimensional's drivers create 3D images by altering the computer's screen, sending the user two different angles of the same image or scene. With our 3D Video Conferencing program we are already providing the two angles using two web cameras. When running our program with their software it is clear to see that it does not work properly. The reason being, we need to control the signals being sent from the IR transmitter. Once gaining the signal and replicating it. You should be able to gain access to the LCD shutter glasses.

We were a little disappointed with the response email that we received because, nearing completion of our project, we found out simply running our 3D Video Conferencing program with the eDimensional drivers and LCD shutter glasses was not that easy. The email that we received from eDimensional, though not correct in explaining how to correctly use our program with their hardware and software, it was still helpful in guiding us to the right direction.

Video

3D Video Conference

Before fully diving into programming this application we first decided to do some small research of different techniques of getting a local video feed from at least one camera. Since we decided to do the project in C# our options were somewhat limited but fortunately there are many tutorials and smart people out there who produced many great things which really helped with different aspects of our project.

First we looked at three main techniques of getting an actual video feed from a webcam to appear in a simple program. These three techniques were:

- Use avicap32.dll
- Use the WIA API
- Use DirectShowNet (many options if this technique is chosen)

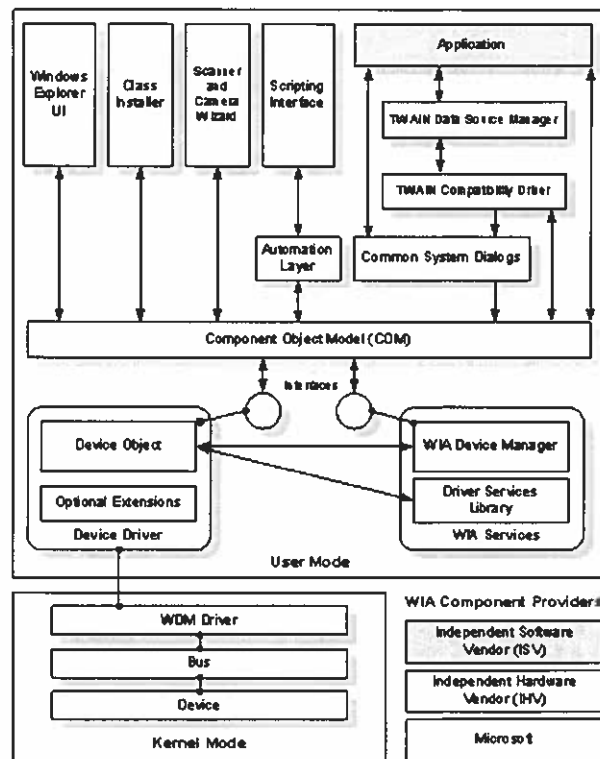
Avicap32.dll

The first technique, using avicap32.dll, we did not spend too much time on because the max frame rate that a sample program was able to achieve was around 15-20 frames/second. We needed ~30 frames/second to be able to sync with the LCD Shutter glasses from eDimensional.

It required using the clipboard to capture frame data which contributed to the max frame rate being on the low side as using the clipboard to copy data from can be slow. There may have been a way to access the image data more directly but we did not stumble upon one. Also, we did not find a way of accessing two webcams with this method, only one.

WIA API

The second technique, using the WIA (Windows Image Acquisition) API, was by far the easiest method to use, however, this would only be compatible in Windows XP because support for video content using the WIA API was removed from WIA for Vista. Here is the WIA architecture:



Not a very complex architecture so it was very easy to understand. We were able to easily enumerate the devices and had a simple program running that would take the first two devices and show their feed in the C# windows form. Showing the video feed was just a matter of passing a handle of a panel or a picture box to a WIA component and the feed would run. A piece of the code is show below to show how to create the video overlay by using the camera device ID and passing the panel or picture box handle:


```

private void CreateVideo(ref System.Windows.Forms.Panel p, string camID)
{
    Cursor.Current = Cursors.WaitCursor;
    wiaVideo = new WiaVideoClass();
    string videoDir = (string)wiaCamera.GetPropById((WiaItemPropertyId)3587);
    wiaVideo.ImagesDirectory = videoDir; // connect video directory

    string cameraID =
        wiaCamera.GetPropById((WiaItemPropertyId)WiaDeviceInfoPropertyId.DeviceI
        nfoDevId) as string;
    wiaVideo.CreateVideoByWiaDevID(camID, p.Handle, 0, 1); // overlay into video panel
}

```

We already knew we had to interleave the frames as one of the goals of our project, so we simply modified the windows form so that one panel/picture box overlapped the other. At around every 33ms we would hide one of the video feeds and show the other. This sort of worked but there was a lot of flickering going on and we knew this was not a viable way to go about this as there was no way to send this over a network because we had no data to send. So now it was a matter of taking frames from at least one camera and being able to store this data anywhere. Unfortunately this seemed to bring up many problems. With a WIA video object, there is a function, `TakePicture()`, and we thought we could use this to capture the frames but this method took too much time. On one of our cameras it took at least three seconds just to grab one frame. We soon realized that the `TakePicture()` function was actually asking the camera to take a picture, render it, and send it back to the program as data. What we really needed was for the program to just be able to look at the video stream and take frame data from the stream itself with the camera doing nothing but running its feed. Having the camera doing the work of capturing frames at 30 fps would probably overwork the camera over a somewhat long period of time. Hit a road block at this point as we could not figure out how to grab frame data and put it into a video never mind sending data over a network. Next was moving on to a different technique.

A previously unmentioned technique: **Webcam.dll**

Before fully diving into DirectShow I stumbled upon a program making use of a Webcam.dll. This too was very easy to use and worked in XP and Vista. This seemed like a great option to use as we were able to send frame data over a local network from one computer to another. We were finally engaged in a form of video conferencing. But then, like the two previous methods, there came a big stumbling block as there was no way to use two devices using the Webcam.dll. It simply took the default capture device and did not allow a programmer to choose his own device. Now it was finally time to dive into DirectShow. A simple piece of code showing the capture of a frame and sending it over the network:

```
Socket udpSocket = new Socket(AddressFamily.InterNetwork, SocketType.Dgram,
    ProtocolType.Udp);
out_ms = new MemoryStream();
p[0].Image.Save(out_ms, System.Drawing.Imaging.ImageFormat.Jpeg);
byte[] buffer = out_ms.GetBuffer();
out_ms.Close();
IPEndPoint iep = new IPEndPoint(IPAddress.Parse(ipAddress), 8080);
udpSocket.SendTo(buffer, iep);
udpSocket.Close();
```

DirectShow

DirectShow proved to be the most flexible and efficient way to go about any type of video conferencing out of all the methods we tried thus far. This would usually be a larger task because the DirectShow interfaces was not fully available for C# but fortunately a DirectShow.NET library was created to bring DirectShow functionality to .NET applications. Going with this method also provided the most tutorials and documentation because a lot of information and samples using C++ could easily be found and converted to C#. Also, we could use as many devices as we wanted as long as our system could handle it. Although this is the most flexible way, it was the most complex way as well but it was not anything that we could not manage. All

the examples referenced were found directly on the DirectShow.NET library website at:
<http://directshownet.sourceforge.net/>

The first sample program we looked at was called CapWMV. This sample program would initialize the default capture device and record the feed to a windows media video file using the ASFWriter DirectShow interface. You can specify the encoding parameters as long as you have the proper GUID. This seemed like a great way to start because it could record in a stream that would be viable for use over the internet but we then again ran into problems. First problem was that in order to use ASFWriter the whole video conference would have to be recorded to one file and if you go on for a long period of time this file can get big. Another problem, a bigger problem, was that we could not get ASFWriter to write frames from two capture devices to one feed. We could easily write two feeds at once but that just leaves us with two separate files. Interleaving frames of two different windows media files would require the use of the windows media encoder SDK which can be a very slow process.

Next example we looked at was a program called DxWebCam which is described, on the site, as a “poor man’s web cam program.” What this program basically did was use the ISampleGrabber interface to capture an image from the device capture graph and it would upload the image to a file server. An image was captured at every tenth of a second and it would be uploaded to the file server using TCP. This looked like it could be very useful because we could maybe take images at a faster interval (every ~33ms) from two different devices and combine these frames into video form. At this point we had to look into different video codec’s.

Video Codec's

First we would have to look in general how different video compress/decompression techniques worked. There was a wealth of information all over the place so it was easy to come by. What we gathered was basically there were two general types and they were intraframe and interframe compression. Intraframe is very simple and it is essentially image compression of each video frame. This type of video compression makes the video easy to edit because you can cut and edit any frame because all frames are there for you to access. Interframe is a very different technique and can achieve very high compression ratios and needs to be used for a video conferencing system if you wanted to use it over the internet.

Interframe compression basically contains three different types of frames, intra coded frames (I Frames), predicted frames (P Frames), and bi-directional predicted frames (B-Frames). I-Frames, or key frames, are frame which do not need to reference any other frame to be built. It already contains all the information it needs to be built. P Frames require a previous frame to be decoded before it can be built. It usually references a key frame. It copies any of the same data from a preceding frame to the P frame and only keeps the new or shifted data of the current frame. B frames can look at previous or future frames to build itself, so it's like a p frame except it can look forward as well instead of only backwards. Depending on implementation this can allow suitable bit rates for the internet. That's only basic information that we decided to look into and then we jumped into different compression/decompression techniques.

We looked at a lab that was given to a computer science graduate class that had to access a video file from a server and the server would have to stream the video to the client. This was all done in java and the codec they used was MJPEG (motion JPEG). This codec would be the easiest to implement since it is pretty much is just a stream of JPEG's put together. This however would

definitely not be suitable for the internet as each frame would essentially be a key frame and at 30 fps the bit rate would be large. You can see why intraframe compression is essentially just image compression as the only compression occurs is the compression of each individual frame of the stream.

Many of the windows media compression techniques could have been suitable for doing this over the internet but since we had individual frames we would have to use windows media encoder SDK to put these frames together and then let the encoder encode the video but this proved to be a very time consuming process which would result in a massive delay on the video feeds when received on the other end.

Although an uncompressed AVI would not be suitable for the internet, we tried it anyway. This had the same problems as MJPEG if we did not compress it at all. Applying any compression brought up the same problems of the windows media encoder dealing with speed. And lastly, AVI was not meant for streaming as it needed all the frames before it could create the video so no appending to it. This would probably require recording in short video slices and then sending which would be a tedious task without much usefulness.

We finally looked at h.264 which is what most video conferencing applications used. This was created jointly by ITU-T Video Coding Experts Group (VCEG) together with the ISO/IEC Moving Picture Experts Group (MPEG). This would be the best option for future development because it's pretty much the standard for video conferencing but before even trying to create a program making use of this there was one major problem, there were no libraries or API's for h.264 functionality in .NET applications. We could have, or should have, looked at a program called VLC media player. It's a free open source program which can play just about any media type and can stream any content to another computer in a network. All the available VLC code is

done in C and although we are doing this in C#, we could have created a wrapper around some of the VLC libraries that would be useful to us. Unfortunately time constraints did not allow us to do this, but this would be a perfect start for future development.

Because we could not use any good compression techniques in C# I decided to try and do something similar to what these codec's were doing. I would set up an interval for a key frame such having a key frame show up every 10 frames or something like that. Once a key frame has been shown I would like to compare the next frame to the previous key frame and only send differences of it. I quickly realized I was trying to invent a sloppy codec and that image processing is very, very, very slow in C# even when using pointers.

We went with the easiest implementation of using MJPEG. Knowing this we were now limited to local network conferencing but that would be fine as this is a stepping stone for future development.

References

<http://blogs.msdn.com/coding4fun/archive/2006/10/31/912546.aspx>

<http://www.videolan.org/vlc/>

<http://www.newmediarepublic.com/dvideo/compression/adv07.html>

http://www.axis.com/documentation/whitepaper/video/compression_standards.htm

http://en.wikipedia.org/wiki/Video_compression#Intraframe_vs_interframe_compression

<http://en.wikipedia.org/wiki/H.264>

http://www.axis.com/documentation/whitepaper/video/compression_standards.htm

Networking

3D Video Conference

To stream media we needed to use a network protocol that would be fast enough to meet our needs. TCP (Transmission Control Protocol) would be too slow although it would guarantee delivery on all packets and in order transmission but speed is a must. So our other option was UDP (User Datagram Protocol) which met our speed needs however it is an unreliable, connectionless protocol. This means that packets sent over a network may arrive out of order, late, or not at all. So would we need to create a reliable layer over UDP? The answer would be no because there is no need to try and reinvent a faster TCP. The thing with media is that it can afford to lose data but we would need to find a way to tolerate any loss data. We would need some type of fault tolerance in our application.

+	Bits 0 - 7	8 - 15	16 - 23	24 - 31
0	Source address			
32	Destination address			
64	Zeros	Protocol	UDP length	
96	Source Port		Destination Port	
128	Length		Checksum	
160	Data			

The structure of a UDP datagram

First, considering the unordered packets problem, we decided to add an integer identifier at the beginning of each packet to identify what packet it is to make sure the order is correct. This was an easy task because all it required was taking the first four bytes of the packet and they would be the index of the packet.

Sending:

```
int dgNum = bImg.Length / 512, remain = bImg.Length % 512;
byte[] rem = new byte[remain + 4];
for (int i = 0; i < dgNum; i++)
{
    System.Buffer.BlockCopy(BitConverter.GetBytes(i), 0, buffer, 0, 4);
    System.Buffer.BlockCopy(bImg, i * 512, buffer, 4, 512);
    udpclient.Send(buffer, buffer.Length);
}
System.Buffer.BlockCopy(BitConverter.GetBytes(dgNum), 0, rem, 0, 4);
System.Buffer.BlockCopy(bImg, dgNum * 512, rem, 4, remain);
udpclient.Send(rem, rem.Length);
```

Receiving:

```
tmp = udpClient.Receive(buffer);
System.Buffer.BlockCopy(buffer, 0, header, 0, 4);
if (BitConverter.ToInt32(header, 0) < 0)
{
    dispImg(imgRcv);
    count = 0;
    break;
}
System.Buffer.BlockCopy(buffer, 4, imgRcv, count * 512, buffer.Length - 4);
count++;
```

This doesn't totally fix the problem but it is one step towards the right direction. To think of the other problem, dropped packets, was a little more difficult. Fortunately for streaming media types, not all data is needed at once for it to become useful data. But what would we do if they were so many missing packets? We tried sending an image over UDP over the internet and we found many dropped packets. How many dropped packets could we tolerate before asking for retransmission of an important packet. A lot of questions that needed answers but at this point since we already know the method of sending frames over the network was not viable for the internet anyway, we started working more on the other parts of the project.

As was discovered when looking at VLC media player code, there is a lot of good information in their code. RTP (Realtime Transport Protocol) was created specifically for the transmission of audio and video over a network and the VLC has many methods for this using h.264 video. So

I'll reiterate that looking at VLC code would be a great starting point as it has many methods that standard video conferencing applications would use.

+ Bits	0-1	2	3	4-7	8	9-15	16-31
0	Ver.	P	X	CC	M	PT	Sequence Number
32	Timestamp						
64	SSRC identifier						
96	... CSRC identifiers ...						
96+(CC×32)	Extension header (optional).						
96+(CC×32) + (X×((EHL+1)×32))	Data						

RTP Packet Structure

References

http://en.wikipedia.org/wiki/User_Datagram_Protocol

http://en.wikipedia.org/wiki/Internet_Protocol

http://en.wikipedia.org/wiki/Real-time_Transport_Protocol

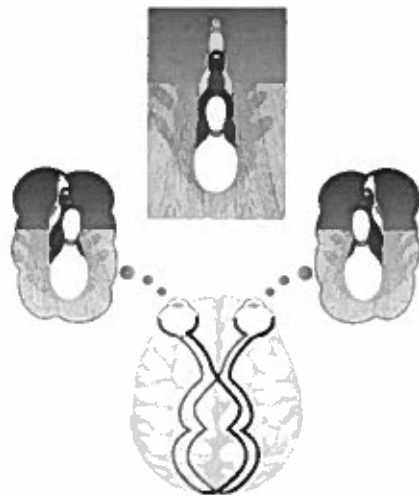
<http://www.videolan.org/vlc/>

Optics

3D Video Conference

Vision

Before starting our project we had to research human vision and how it works. Most of all humans are born, each, with a pair of eyes. Each eye sees the same exact area as the other eye but at a different angle. Our brains combine these two images from the eyes and perceive the differences between the two as “depth.”

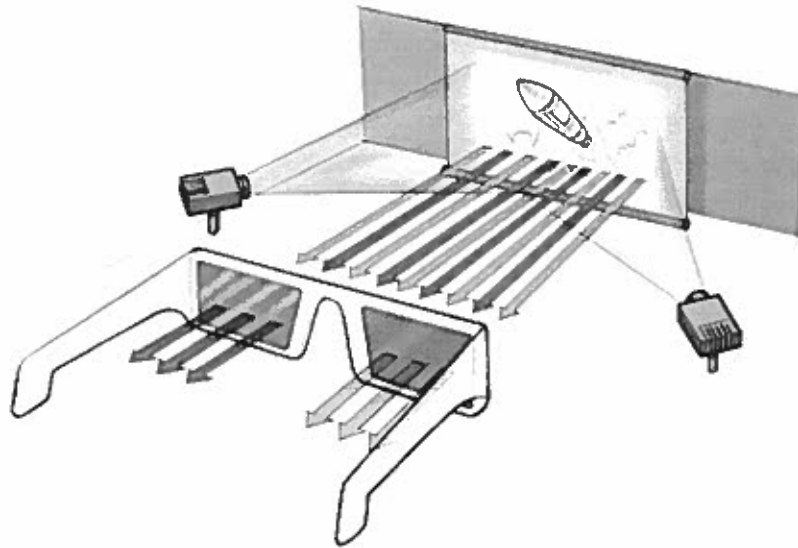


Without depth perception we would have problems seeing three-dimensional objects and images. In some cases, some of the things we might see could be perceived as two-dimensional. People who cannot see three-dimensional images or scenes do not have well established “binocular vision.” Binocular vision is when both eyes are working together equally and simultaneously creating smooth and accurate results. People who do not have strong binocular vision can get their vision surgically corrected or they can simply strengthen their binocular vision with short exercises. Binocular vision is “key” in seeing images and scenes that are three-dimensional.

3D Methods

Paper 3D Glasses

The paper 3D glasses are the more “traditional” means of projecting a 3D image. The reason it's said to be “traditional” is because older films and television shows used this method for 3D before moving on to more sophisticated and elaborate methods.

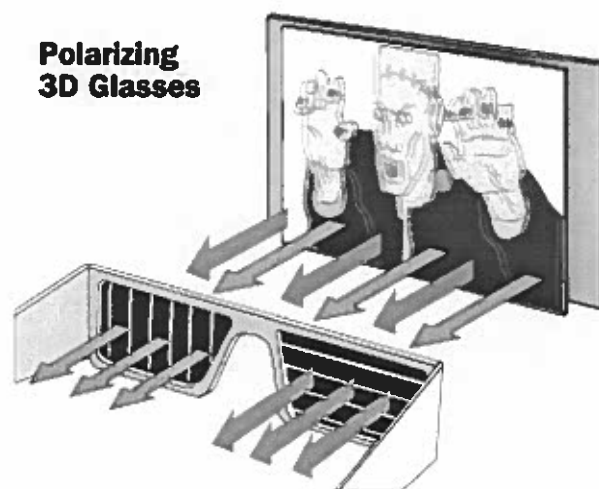


Paper 3D glasses were popular because they were easy to make and obtain. It's just a paper cutout of a shape that is to be comfortable to the people using it. It also has red and blue (or green) colored filters covering each eye. How these glasses work is, for instance, if there is a specific scene or image that it on the screen, each eye takes in a specific color. There are two projectors, one being red and the other being blue. Each projector is projecting the same image or scene but at a different angle and color. So the eye that is covered by the blue filter would take in the red image because the brain “blocks” out the blue and takes in the rest. For the red filter, it does the same thing as the blue side but instead the brain takes in the blue color while blocking out the red image. The pros for using this method of 3D, of course, are its cheap price and acquisition. The con for using it is that you do not get a clear image. The specific scene or

image can be 3D but it is a mix of red and blue colors so you do not actually get the full quality and experience of a “true” 3D object. The scenes and images using the paper 3D method are limited in movement. Because of the two colored projectors, they work best with still photography and minimal movement. Other methods of 3D can handle movement and quality better.

Polarized Glasses

Polarized glasses are used in such places like IMAX Theaters and theme parks like Universal Studios and Disney. A polarized image or scene is created similarly to the paper 3D method. There are two projectors set up to run simultaneously at slightly different angles. Instead of there being different colored projectors, the two projects are projecting at different polarizations. Through the orthogonal polarizing filters from the glasses, the superimposed images from the two polarized projectors create the illusion of 3D. What each filter on the polarized glasses does is that it blocks out the light from the other (orthogonal) projection and takes the rest of the superimposed image in to the brain. The same would go for the other filter covering the other eye.



The figure above displays how this method of using polarized images to create the illusion of 3D works. Many theaters use this method because of its quality. With the red and blue filtered glasses, the color will probably not match the natural color of the original object or image. With polarized glasses you do not have to sacrifice the color of the image because the polarized projectors are of similar color to the original object it is going to be representing.

LCD Shutter Glasses

With the eDimensional LCD shutter glasses the quality of the 3D objects are good. Using the glasses along with the provided eDimensional drivers, dongle and IR transmitter, and the user can experience a much more “interactive” experience with their PC video games. What the dongle does is that it connects straight to the user’s computer. The VGA connector then connects to the dongle along with the IR transmitter. The dongle is the main factor for eDimensional’s way of 3D. This dongle creates two images at different angles constantly flipping between each other at a very fast rate (around 30 frames-per-second). With the LCD shutter glasses correctly synced with the drivers, the shutter glasses also flip between what images are allowed for the user to see and which are not. Because our eyes each see the same image but at different angles, using this knowledge combined with the constant flipping of the images on the screen at a high rate it creates the illusion of 3D.



The figure above is the wireless version of eDimensional's 3D Glasses. The wired version is directly connected to the dongle and does not use an IR transmitter to tell the glasses which LCD screens to shut off.

Afterwards

We realized what eDimensional's drivers were doing. It creates two angles of the image for the user to take in. With our program we are already providing the angles from the web cameras.

From the email we received from eDimensional, we were under the impression that all we needed to do to create a 3D image was to have our program run with their LCD shutter glasses and drivers. We did not realize soon enough that we needed to gain access to the IR transmitter or use an IR emitter and send out the replicated signal to the shutter glasses. Gaining the signal from the IR transmitter from the dongle provided would enhance the 3D experience greatly.

Correctly syncing the glasses and the program together would create the illusion of a 3D scene or image.

References

3D Video Conference

<http://www.vision3d.com/frame.html>

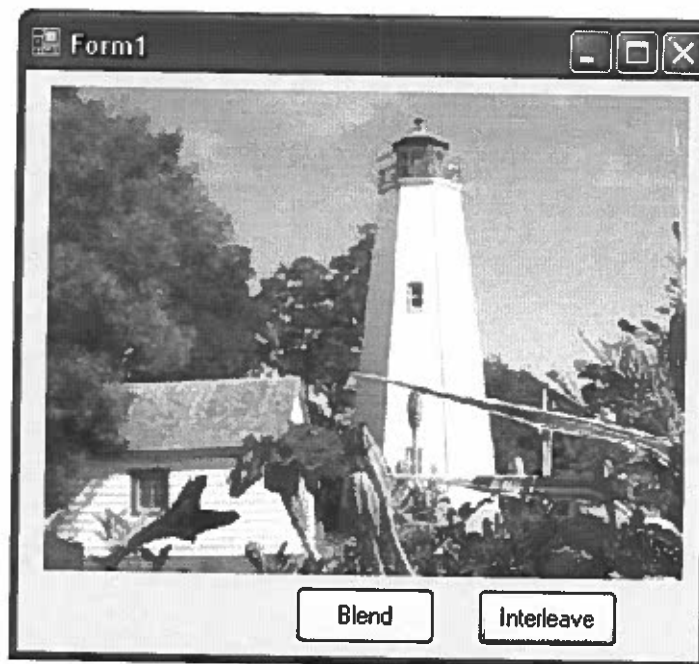
<http://www.3dglassesonline.com/how-do-3d-glasses-work/>

<http://en.wikipedia.org/wiki/Stereoscopic>

<http://edimensional.com/>

Interleaving of 2 Images at 30 frames/second

This piece of code was made just to see if two images can be interleaved fast enough. There was a little flickering although we cleared that up a little later by enabling a double buffering technique.



```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Drawing.Drawing2D;
using System.Drawing.Imaging;
using System.Text;
using System.Windows.Forms;
using WIA;
using WIAVIDEOLib;

namespace webcam2
{
    public partial class Form1 : Form
    {
        private Device[] dev = new Device[2];
        private string jpegGuid = string.Empty;
        static int height = 240, width = 320;
        static int count = 0;
        static PictureBox pBox;
        static System.Windows.Forms.Timer myTimer = new System.Windows.Forms.Timer();
        static Control mainForm = null;
        static bool exitFlag = false;
        static Image img1 = Image.FromFile("1.jpg");
        static Image img2 = Image.FromFile("2.jpg");
        static Bitmap b1;
        static Bitmap b2;

        public Form1()
        {
            InitializeComponent();
            pBox = pictureBox1;
            b1 = new Bitmap(img1);
            b2 = new Bitmap(img2);
            mainForm = TopLevelControl;
        }

        private void Form1_Load(object sender, EventArgs e)
        {
            /*Image img1 = Image.FromFile("1.jpg");
            Image img2 = Image.FromFile("2.jpg");
            pictureBox1.Image = img1;*/
        }

        private void button1_Click(object sender, EventArgs e)
        {
            /*Image img1 = Image.FromFile("1.jpg");
            Image img2 = Image.FromFile("2.jpg");*/
            //Bitmap b1 = new Bitmap(img1);
            //Bitmap b2 = new Bitmap(img2);
            //BitmapData b1data = b1.LockBits(new Rectangle(0, 0, width, height),
            ImageLockMode.ReadWrite, PixelFormat.Format24bppRgb);
            //BitmapData b2data = b2.LockBits(new Rectangle(0, 0, width, height),
            ImageLockMode.ReadWrite, PixelFormat.Format24bppRgb);

            //unsafe
            //{
            //    byte* dst = (byte*)(void*)b1data.Scan0;
            //    byte* src = (byte*)(void*)b2data.Scan0;
```

```

        //      for (int y = 0; y < height; y++)
        //      {
        //          for (int x = 0; x < bldata.Stride; x++)
        //          {
        //              /*if (dst[0] < 128)
        //              {
        //                  dst[0] = (byte)Math.Max(Math.Min((src[0] / 255.0f * dst[0]
/ 255.0f) * 255.0f * 2, 255), 0);
        //              }
        //              else
        //              {
        //                  dst[0] = (byte)Math.Max(Math.Min(255 - ((255 - src[0]) /
255.0f * (255 - dst[0]) / 255.0f) * 255.0f * 2, 255), 0);
        //              }*/
        //              dst[0] = (byte)Math.Max(Math.Min((src[0] / 255.0f * dst[0] /
255.0f) * 255.0f, 255), 0);
        //              // dst[0] = (byte)Math.Max(Math.Min(255 - ((255 - src[0]) / 255.
0f * (255 - dst[0]) / 255.0f) * 255.0f, 255), 0);
        //              dst++;
        //              src++;
        //          }
        //      }

        //b1.UnlockBits(bldata);
        //b2.UnlockBits(b2data);

        //pictureBox1.Image = b1;

```

```

    }

```

```

private void MainForm_Paint(object sender, System.Windows.Forms.PaintEventArgs e)
{
    Graphics g = e.Graphics;
    g.DrawImage(b1, 0, 0, width, height);
}

```

```

private static void TimerEventProcessor(Object myObject,
                                         EventArgs myEventArgs)
{

```

```

    count = 1 - count;
    if (!exitFlag)
    {
        if (count == 1)
        {
            pBox.Image = b1;
        }
        else
        {
            pBox.Image = b2;
        }
        //mainForm.Paint += new PaintEventHandler(MainForm_Paint);
    }
    else
        myTimer.Enabled = false;
}

```

```

private void button2_Click(object sender, EventArgs e)
{
    myTimer.Tick += new EventHandler(TimerEventProcessor);
}

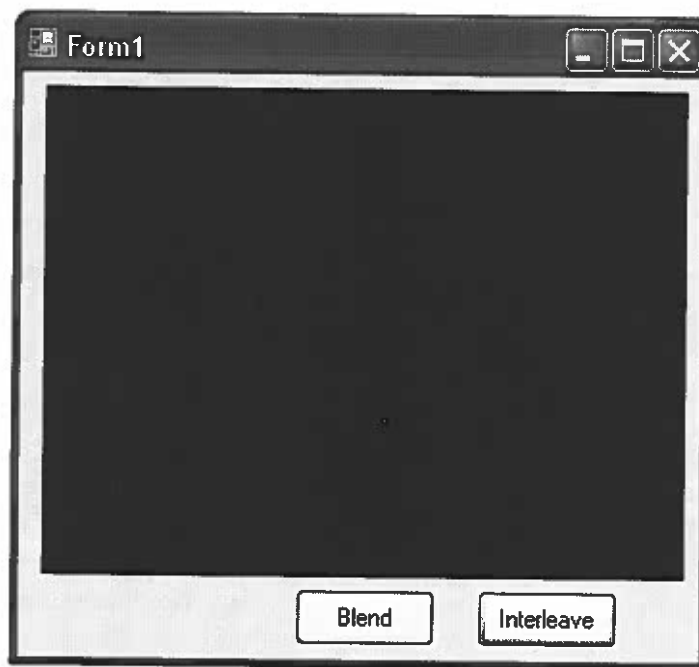
```

```
myTimer.Interval = 1;
myTimer.Start();

// Runs the timer, and raises the event.
while (exitFlag == false)
{
    // Processes all the events in the queue.
    Application.DoEvents();
}
}
}
```

Interleaving of 2 video feeds at 30 frames/second

I altered the previous program to interleave to two WIA Video feeds instead of simple images and while it was able to interleave fast enough there was a lot of flickering and double buffering would not help solve this problem.



```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Drawing.Drawing2D;
using System.Drawing.Imaging;
using System.Text;
using System.Windows.Forms;
using WIA;
using WIAVIDEOLib;
using QuartzTypeLib;

namespace WindowsApplication2
{
    public partial class Form1 : Form
    {
        private IBasicVideo2 bv;

        private DateTime start;
        private WiaVideoClass[] wiaVideo = new WiaVideoClass[2];
        private Device[] dev = new Device[2];
        static Panel[] panel = new Panel[2];
        private int panel_index;
        private int height = 240, width = 320;
        static int count = 0;
        private bool stop = true;
        static System.Windows.Forms.Timer myTimer = new System.Windows.Forms.Timer();
        static bool exitFlag = false;

        WindowsApplication2.Properties.Settings settings = WindowsApplication2.Properties.
        .Settings.Default;
        public Form1()
        {
            InitializeComponent();
            panel_index = 0;
            panel[0] = vidPanel;
            panel[1] = vidPanel2;
        }

        private void Form1_Load(object sender, EventArgs e)
        {
            //WIA Device Manager
            DeviceManager manager = new DeviceManagerClass();
            //string did="";
            //enumerate WIA devices
            foreach (DeviceInfo info in manager.DeviceInfos)
            {
                //left cam
                /*if (info.DeviceID == "{6BDD1FC6-810F-11D0-BEC7-08002BE2092F}\\0003")
                {
                    camidlab.Text = info.DeviceID;
                    dev = info.Connect();
                    CreateVideo(ref panel[panel_index++], info.DeviceID);
                }*/
                /*if (panel_index == 0)
                {
                    did = info.DeviceID;
                    dev = info.Connect();
                    CreateVideo(ref panel[panel_index++], info.DeviceID);
                }*/
            }
            //right cam
        }
    }
}
```

```

        //else if (info.DeviceID == "{6BDD1FC6-810F-11D0-BEC7-08002BE2092F}\\0007"
    ")
        //if (info.DeviceID != did)
        //{
        if (info.Type.ToString() == "VideoDeviceType")
        {
            dev[panel_index++] = info.Connect();
            //dev[0].DeviceID;
            //MessageBox.Show(info.DeviceID);
            //CreateVideo(ref panel[panel_index++], info.DeviceID);
        }
        //}

        //both panels now occupied
        if (panel_index == 2) break;
    }
    CreateVideo(ref panel[0], "hiyoo");
}

private void CreateVideo(ref System.Windows.Forms.Panel p, string camID)
{
    wiaVideo[0] = new WiaVideoClass();
    wiaVideo[1] = new WiaVideoClass();
    wiaVideo[0].ImagesDirectory = "c:\\tmp2";
    wiaVideo[0].CreateVideoByWiaDevID(dev[0].DeviceID, panel[0].Handle, 1, 1);
    wiaVideo[1].ImagesDirectory = "c:\\tmp";
    wiaVideo[1].CreateVideoByWiaDevID(dev[1].DeviceID, panel[1].Handle, 1, 1);

    //MessageBox.Show(start.ToString());
}

private static void TimerEventProcessor(Object myObject,
                                         EventArgs myEventArgs)
{
    count = 1 - count;
    if (!exitFlag)
    {
        if (count == 1)
        {
            panel[1].Visible = false;
            panel[0].Visible = true;
        }
        else
        {
            panel[0].Visible = false;
            panel[1].Visible = true;
        }
    }
    else
        myTimer.Enabled = false;
}

private void button1_Click(object sender, EventArgs e)
{
    /*string jpgFile;
    // DisposePicture();

    wiaVideo.TakePicture(out jpgFile);           // call IWiaVideo::
TakePicture
    //MessageBox.Show(jpgFile);

    if (jpgFile != null)

```



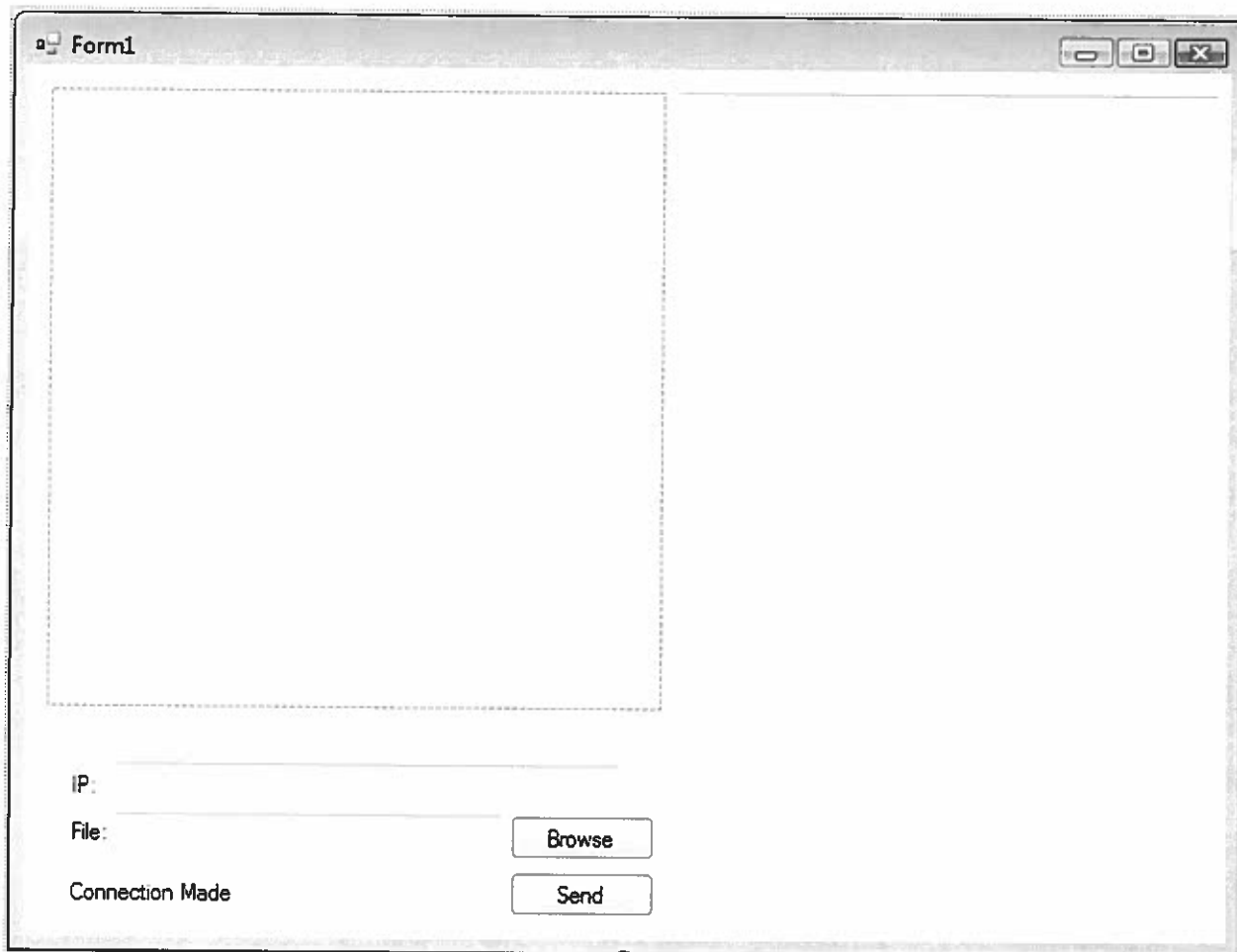
```
    {
        pictureBox1.Image = Image.FromFile(jpgFile);    // load jpeg file
    }
    into preview
    }*/
    //for (int i = 0; i < 100; i++)
    //{
        //    for (int j = 0; j < 1000000; j++)
        //        for (int k = 0; k < 100; k++) ;
        //    count = 1 - count;
        //    if (count == 1)
        //    {
        //        panel[1].Visible = false;
        //        panel[0].Visible = true;
        //    }
        //    else
        //    {
        //        panel[0].Visible = false;
        //        panel[1].Visible = true;
        //    }
    //}
    myTimer.Tick += new EventHandler(TimerEventProcessor);
    myTimer.Interval = 33;
    myTimer.Start();

    // Runs the timer, and raises the event.
    while (exitFlag == false)
    {
        // Processes all the events in the queue.
        Application.DoEvents();
    }
}

private void button2_Click(object sender, EventArgs e)
{
    stop = false;
    //MessageBox.Show((DateTime.Now - start).TotalMilliseconds.ToString());
    exitFlag = true;
    myTimer.Enabled = false;
}
}
```

“UIM” Program: Sending a picture over UDP

This was a simple program using a UDP Socket to send a picture over the network.



```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Windows.Forms;
using System.Net;
using System.Net.Sockets;
using System.IO;
using System.Threading;
using System.Collections;
using System.Drawing.Imaging;

```

```

namespace UIM

```

```

{
    public partial class Form1 : Form
    {
        public static ArrayList alSockets;
        public string img_file;
        private static Object thisLock1 = new Object();
        public static string hostname = Dns.GetHostName();
        public IPHostEntry IPHost = Dns.GetHostEntry(hostname);
        delegate void dispMsgCallback(string msg);

        public Form1()
        {
            InitializeComponent();
        }

        public void receivingThread()
        {
            //create socket for udp connection
            Socket udpSocket = new Socket(AddressFamily.InterNetwork,
SocketType.Dgram, ProtocolType.Udp);
            IPEndPoint ipepServer = new IPEndPoint(IPAddress.Any, 8602);
            udpSocket.Bind(ipepServer);
            //listenerSocket.Listen(-1);
            int byteReceived = 0;
            byte[] buffer = new byte[260000];
            byte[] buffer2 = new byte[516];
            byte[] header = new byte[4];
            int count = 0;

            while (true)
            //do
            {
                ArrayList bufferAL;

                byteReceived = udpSocket.Receive(buffer2);
                System.Buffer.BlockCopy(buffer2, 0, header, 0, 4);

                //String msg = System.Text.Encoding.ASCII.GetString
(buffer);

                //dispMsg(msg);
                //if (byteReceived > 0)
                //    dispMsg(count.ToString());
            }
        }
    }
}

```

```

        //if ((int)buffer2[0] < 0)
        if (BitConverter.ToInt32(header, 0) < 0)
        {
            //dispMsg("i got in");
            tryF(buffer);
            count = 0;
            Array.Clear(buffer, 0, buffer.Length);
        }
        else
        {
            //tryF(buffer);
            //dispMsg(BitConverter.ToInt32(header, 0).ToString
            System.Buffer.BlockCopy(buffer2, 4, buffer, count *
            count++;
        }

        //textBox3.Text += msg + "\n";

        //buffer[count] = recv[0];
        //count++;
        //textBox1.Text = "yup";
    } //while (count < 1300);
    //textBox1.Text += "got";
    tryF(buffer);
}
public void dispMsg(string msg)
{
    if (this.textBox3.InvokeRequired)
    {
        dispMsgCallback d = new dispMsgCallback(dispMsg);
        this.Invoke(d, new object[] { msg });
    }
    else
    {
        this.textBox3.Text += msg + "\n";
    }

    //textBox3.Text += msg + "\n";
}

public void tryF(byte[] b)
{
    pictureBox1.Image = convert_byte(b);
    //textBox2.Text += " it";
}

private byte[] convert_img(Image img)
{
    MemoryStream ms = new MemoryStream();
    img.Save(ms, ImageFormat.Jpeg);
    return ms.ToArray();
}
private static Image convert_byte(byte[] b)

```

```

    {
        MemoryStream ms = new MemoryStream(b);
        return Image.FromStream(ms);
    }

private void btn_send_Click(object sender, EventArgs e)
{
    Image img = Image.FromFile(@img_file);
    string ipAddress = textBox1.Text;
    int portNumber = 8602;
    //int portNumber = 8602;
    byte[] buffer = convert_img(img);
    byte[] b2 = new byte[516];
    byte[] msg = new byte[256];
    msg = System.Text.Encoding.ASCII.GetBytes
(textBox2.Text.ToCharArray());
    /*for (int i = 0; i < 513; i++)
    {
        b2[i] = buffer[i];
    }*/
    //textBox1.Text = buffer.Length.ToString();

    UdpClient udpclient = new UdpClient(ipAddress, portNumber);
    //udpclient.Send(buffer, buffer.Length);
    int count = 0, remain, dgNum;
    //textBox1.Text += buffer.Length.ToString();
    if (buffer.Length > 515)
    {
        ArraySegment<byte> byteSeg;
        dgNum = buffer.Length/512;
        remain = buffer.Length % 512;
        byte[] rem = new byte[remain + 4];
        for (int i = 0; i < dgNum; i++)
        {
            //b2[0] = (byte)i;
            System.Buffer.BlockCopy(BitConverter.GetBytes(i), 0,
b2, 0, 4);
            //System.Buffer.BlockCopy(buffer, i * 512, b2, 4,
512);
            byteSeg = new ArraySegment<byte>(buffer, i*512,
512);

            udpclient.Send(b2, b2.Length);
        }
        System.Buffer.BlockCopy(BitConverter.GetBytes(dgNum), 0,
rem, 0, 4);
        System.Buffer.BlockCopy(buffer, dgNum * 512, rem, 4,
remain);
        udpclient.Send(rem, rem.Length);
    }
    int endint = -1;
    byte[] end = BitConverter.GetBytes(endint);
    //char endch = 'e';
    //end[0] = endch.;
    textBox3.Text = end.Length.ToString();

    udpclient.Send(end, end.Length);
}

```

```

        //udpclient.Send(b2, b2.Length);
        //udpclient.Send(msg, msg.Length);

    }

    private void Form1_Load(object sender, EventArgs e)
    {
        alSockets = new ArrayList(); //puts all the created sockets
into an array

        //call creates a listener thread to listen for connections
        Thread recvThread = new Thread(new ThreadStart
(receivingThread));
        recvThread.Start();
    }

    private void button1_Click(object sender, EventArgs e)
    {
    }

    private void button1_Click_1(object sender, EventArgs e)
    {
        OpenFileDialog fd = new OpenFileDialog();
        fd.Multiselect = false;
        fd.Title = "Select PictureBox (jpeg)";
        fd.Filter = "jpeg files (*.jpg; *.jpeg)|*.jpg; *.jpeg";

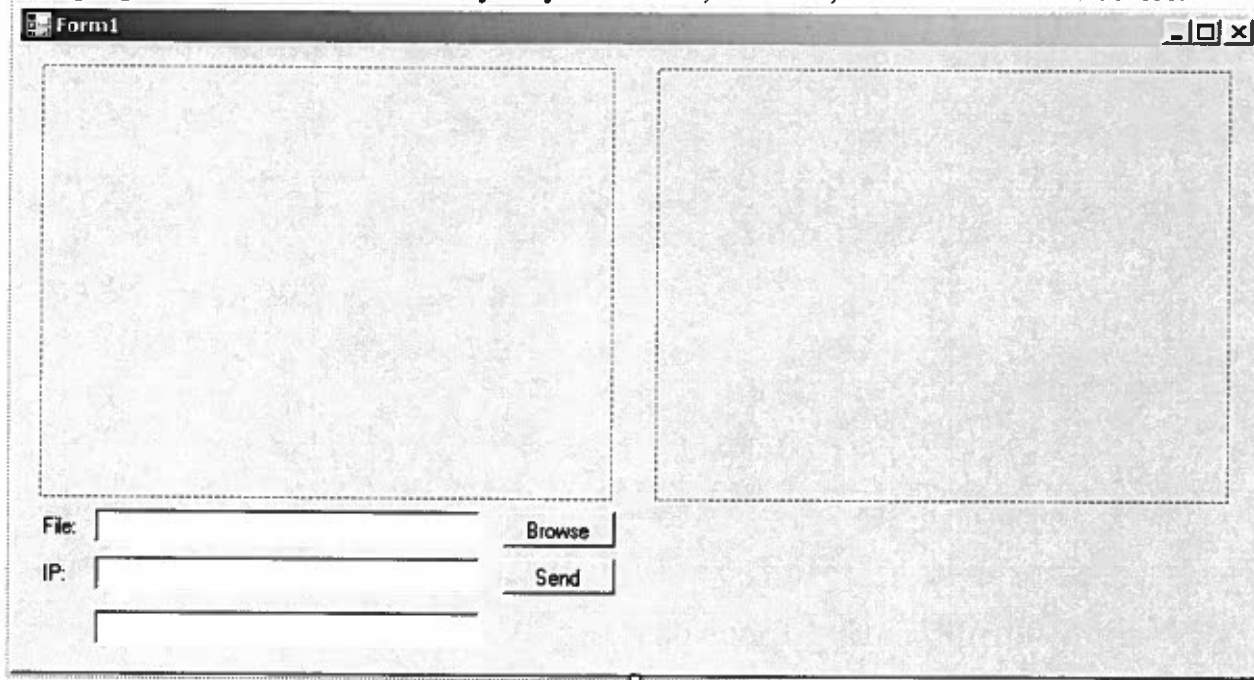
        DialogResult Result = fd.ShowDialog();
        fd.InitialDirectory = @"c:\";

        ///if we get a dir, put path in textbox
        if (Result == DialogResult.OK)
        {
            textBox2.Text = fd.FileName;
            img_file = fd.FileName;
        }
        // Cancel button was pressed.
        else if (Result == DialogResult.Cancel)
        {
            return;
        }
    }
}
}

```

“SendPic” Program: 30 frames/second video conferencing

This program used Webcam.dll. Very easy API to use, however, it limited us to one device.



```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Windows.Forms;
using System.Net;
using System.Net.Sockets;
using System.IO;
using System.Threading;
using System.Collections;
using System.Drawing.Imaging;
using WebCam_Capture;

namespace SendPic
{
    public partial class Form1 : Form
    {
        //private WebCamCapture[] cam = new WebCamCapture[2];
        private WebCamCapture[] cam = new WebCamCapture[1];
        static PictureBox[] p = new PictureBox[1];
        //static PictureBox[] p = new PictureBox[1];
        static System.Windows.Forms.Timer myTimer = new
System.Windows.Forms.Timer();
        static private MemoryStream in_ms, out_ms;
        static Control mainForm = null;
        //static UdpClient udpclient = new UdpClient("192.168.10.103",
8080);
        static bool exitFlag = false;
        static private string ipAddress;

        public static string hostname = Dns.GetHostName();
        public IPHostEntry IPHost = Dns.GetHostEntry(hostname);
        public int outport = 8080, inport = 8080;
        public string img_file;
        delegate void dispMsgCallback(string msg);
        delegate void dispPicCallback(MemoryStream ms);

        public Form1()
        {
            InitializeComponent();
            mainForm = TopLevelControl;
        }

        public void receivingThread()
        {
            // Socket udpClient = new Socket(AddressFamily.InterNetwork,
SocketType.Dgram, ProtocolType.Udp);
            // //UdpClient udpClient = new UdpClient(inport);
            // IPEndPoint remEP = new IPEndPoint(IPAddress.Any, inport);
            // udpClient.Bind(remEP);

            // byte[] size = new byte[4];
            // byte[] header = new byte[4];
            // byte[] buffer = new byte[516];

```



```

//          int count = 0, tmp;

//          while (true)
//          {
//              byte[] imgRcv = new byte[35000]; //BitConverter.ToInt32
//              (size, 0) + 4];

//              while (true)
//              {
//                  buffer = udpClient.Receive(ref remEP);
//                  tmp = udpClient.Receive(buffer);

//                  System.Buffer.BlockCopy(buffer, 0, header, 0, 4);

//                  if (BitConverter.ToInt32(header, 0) < 0)
//                  {
//                      dispImg(imgRcv);
//                      count = 0;
//                      break;
//                  }
//                  //dispMsg((count.ToString() + " "));
//                  System.Buffer.BlockCopy(buffer, 4, imgRcv, count *
512, buffer.Length - 4);
//                  count++;
//              }

//              imgRcv = null;
//          }

```

```

Socket udpClient = new Socket(AddressFamily.InterNetwork,
SocketType.Dgram, ProtocolType.Udp);
IPEndPoint remEP = new IPEndPoint(IPAddress.Any, inport);
udpClient.Bind(remEP);
byte[] buffer = new byte[16384];

```

```

while (true)
{
    in_ms = new MemoryStream();
    udpClient.Receive(buffer);
    in_ms.Write(buffer, 0, buffer.Length);
    //pictureBox1.Image = Image.FromStream(in_ms);
    dispPic(in_ms);
    //in_ms.Flush();
}

```

```

}

```

```

public void dispImg(byte[] b)
{
    pictureBox1.Image = convert_byte(b);
}
private byte[] convert_img(Image img)
{
    MemoryStream ms = new MemoryStream();
    img.Save(ms, ImageFormat.Jpeg);
    return ms.ToArray();
}

```

```

    }
    private static Image convert_byte(byte[] b)
    {
        MemoryStream ms = new MemoryStream(b);
        return Image.FromStream(ms);
    }

    private void btn_browse_Click(object sender, EventArgs e)
    {
        OpenFileDialog fd = new OpenFileDialog();
        fd.Multiselect = false;
        fd.Title = "Select PictureBox (jpeg)";
        fd.Filter = "jpeg files (*.jpg; *.jpeg)|*.jpg; *.jpeg";

        DialogResult Result = fd.ShowDialog();
        fd.InitialDirectory = @"c:\\";

        ///if we get a dir, put path in textbox
        if (Result == DialogResult.OK)
        {
            filePathBox.Text = fd.FileName;
            img_file = fd.FileName;
        }
        // Cancel button was pressed.
        else if (Result == DialogResult.Cancel)
        {
            return;
        }
    }

    private static void TimerEventProcessor(Object myObject,
                                           EventArgs myEventArgs)
    {
        if(!exitFlag)
        {
            Socket udpSocket = new Socket
(AddressFamily.InterNetwork, SocketType.Dgram, ProtocolType.Udp);
            out_ms = new MemoryStream();
            p[0].Image.Save(out_ms,
System.Drawing.Imaging.ImageFormat.Jpeg);

            byte[] buffer = out_ms.GetBuffer();
            out_ms.Close();
            //out_ms.Flush();

            IPEndPoint iep = new IPEndPoint(IPAddress.Parse
(ipAddress), 8080);
            udpSocket.SendTo(buffer, iep);
            udpSocket.Close();
        }
        else
            myTimer.Enabled = false;
    }

```

```

private void btn_send_Click(object sender, EventArgs e)
{
    //Image img = Image.FromFile(@img_file);
    //string ipAddress = ipBox.Text;

    //byte[] bImg = convert_img(img);
    //byte[] size = BitConverter.GetBytes(bImg.Length);
    //byte[] buffer = new byte[516];
    //byte[] msg = new byte[256];

    //UdpClient udpclient = new UdpClient(ipAddress, outport);

    ////udpclient.Send(size, 0);

    //int dgNum = bImg.Length / 512,
    //    remain = bImg.Length % 512;
    //byte[] rem = new byte[remain + 4];
    //for (int i = 0; i < dgNum; i++)
    //{
        //    System.Buffer.BlockCopy(BitConverter.GetBytes(i), 0,
buffer, 0, 4);
        //    System.Buffer.BlockCopy(bImg, i * 512, buffer, 4,
512);

        //    udpclient.Send(buffer, buffer.Length);
    //}
    //System.Buffer.BlockCopy(BitConverter.GetBytes(dgNum), 0,
rem, 0, 4);
    //System.Buffer.BlockCopy(bImg, dgNum * 512, rem, 4,
remain);
    //udpclient.Send(rem, rem.Length);

    //int endint = -1;
    //byte[] end = BitConverter.GetBytes(endint);
    //udpclient.Send(end, end.Length);

    ipAddress = ipBox.Text;
    myTimer.Tick += new EventHandler(TimerEventProcessor);
    myTimer.Interval = 35;
    myTimer.Start();

    // Runs the timer, and raises the event.
    while (exitFlag == false)
    {
        // Processes all the events in the queue.
        Application.DoEvents();
    }

    //UdpClient udpclient = new UdpClient("192.168.10.106",
outport);
    //while (true)
    //{
        //    out_ms = new MemoryStream();
        //    pictureBox2.Image.Save(out_ms,
System.Drawing.Imaging.ImageFormat.Jpeg);

        //    byte[] buffer = out_ms.GetBuffer();
        //    //out_ms.Close();
    //}
}

```

```

        //    //out_ms.Flush();

        //    udpclient.Send(buffer, buffer.Length);
        //}

    }

    private void Form1_Load(object sender, EventArgs e)
    {
        cam[0] = new WebCamCapture();
        cam[0].ImageCaptured += new WebCamCapture.WebCamEventHandler
(FrameCap);
        cam[0].TimeToCapture_milliseconds = 33;
        cam[0].Start(0);
        p[0] = pictureBox2;

        Thread re-vThread = new Thread(new ThreadStart
(receivingThread));
        recvThread.Start();
    }

    public void dispMsg(string msg)
    {
        if (this.textBox3.InvokeRequired)
        {
            dispMsgCallback d = new dispMsgCallback(dispMsg);
            this.Invoke(d, new object[] { msg });
        }
        else
        {
            this.textBox3.Text += msg + "\n";
        }

        //textBox3.Text += msg + "\n";
    }

    public void dispPic(MemoryStream ms)
    {
        if (this.pictureBox1.InvokeRequired)
        {
            dispPicCallback d = new dispPicCallback(dispPic);
            this.Invoke(d, new object[] { ms });
        }
        else
        {
            this.pictureBox1.Image = Image.FromStream(ms);
        }

        //textBox3.Text += msg + "\n";
    }

    static void FrameCap(object source, WebcamEventArgs e)
    {

```

```
p[0].Image = e.WebCamImage;
```

```
}
```

```
}
```

```
}
```

Example of using DirectShowNET to write to windows media

This can be tweaked to just grabbed single frames.

```
using System;
using System.Runtime.InteropServices;

using DirectShowLib;

namespace AsfFilter
{
    /// <summary> Summary description for MainForm. </summary>
    internal class Capture: IDisposable
    {
        #region Member variables

        /// <summary> graph builder interface. </summary>
        private IFilterGraph2 m_FilterGraph = null;
        IMediaControl m_mediaCtrl = null;

        /// <summary> Set by async routine when it captures an image </summary>
        private bool m_bRunning = false;

        #if DEBUG
            DsROTEEntry m_rot = null;
        #endif

        #endregion

        /// <summary> release everything. </summary>
        public void Dispose()
        {
            GC.SuppressFinalize(this);
            CloseInterfaces();
        }

        ~Capture()
        {
            Dispose();
        }

        /// <summary>
        /// Create capture object
        /// </summary>
        /// <param name="iDeviceNum">Zero based index of capture device</param>
        /// <param name="szFileName">Output ASF file name</param>
        public Capture(int iDeviceNum, string szOutputFileName)
        {
            DsDevice [] capDevices;

            // Get the collection of video devices
            capDevices = DsDevice.GetDevicesOfCat( FilterCategory.VideoInputDevice );

            if (iDeviceNum + 1 > capDevices.Length)
            {
                throw new Exception("No video capture devices found at that index!");
            }

            try
            {
                // Set up the capture graph
                SetupGraph( capDevices[iDeviceNum], szOutputFileName);

                m_bRunning = false;
            }
            catch
            {
                Dispose();
                throw;
            }
        }
    }
}
```

```

    }
}

// Start the capture graph
public void Start()
{
    if (!m_bRunning)
    {
        int hr = m_mediaCtrl.Run();
        Marshal.ThrowExceptionForHR( hr );

        m_bRunning = true;
    }
}

// Pause the capture graph.
// Running the graph takes up a lot of resources. Pause it when it
// isn't needed.
public void Pause()
{
    if (m_bRunning)
    {
        IMediaControl mediaCtrl = m_FilterGraph as IMediaControl;

        int hr = mediaCtrl.Pause();
        Marshal.ThrowExceptionForHR( hr );

        m_bRunning = false;
    }
}

/// <summary> build the capture graph. </summary>
private void SetupGraph(DsDevice dev, string szOutputFileName)
{
    int hr;

    IBaseFilter capFilter = null;
    IBaseFilter asfWriter = null;
    ICaptureGraphBuilder2 capGraph = null;

    // Get the graphbuilder object
    m_FilterGraph = (IFilterGraph2)new FilterGraph();

#if DEBUG
    m_rot = new DsROTEEntry( m_FilterGraph );
#endif

    try
    {
        // Get the ICaptureGraphBuilder2
        capGraph = (ICaptureGraphBuilder2) new CaptureGraphBuilder2();

        // Start building the graph
        hr = capGraph.SetFiltergraph( m_FilterGraph );
        Marshal.ThrowExceptionForHR( hr );

        // Add the capture device to the graph
        hr = m_FilterGraph.AddSourceFilterForMoniker(dev.Mon, null, dev.Name, out
capFilter);
        Marshal.ThrowExceptionForHR( hr );

        asfWriter = ConfigAsf(capGraph, szOutputFileName);

        hr = capGraph.RenderStream(null, null, capFilter, null, asfWriter);
        Marshal.ThrowExceptionForHR( hr );
    }
}

```



```

        m_mediaCtrl = m_FilterGraph as IMediaControl;
    }
    finally
    {
        if (capFilter != null)
        {
            Marshal.ReleaseComObject(capFilter);
            capFilter = null;
        }
        if (asfWriter != null)
        {
            Marshal.ReleaseComObject(asfWriter);
            asfWriter = null;
        }
        if (capGraph != null)
        {
            Marshal.ReleaseComObject(capGraph);
            capGraph = null;
        }
    }
}

private IBaseFilter ConfigAsf(ICaptureGraphBuilder2 capGraph, string
szOutputFileName)
{
    IFileSinkFilter pTmpSink = null;
    IBaseFilter asfWriter = null;

    int hr = capGraph.SetOutputFileName( MediaSubType.Asf, szOutputFileName, out
asfWriter, out pTmpSink);
    Marshal.ThrowExceptionForHR( hr );

    try
    {
        IConfigAsfWriter lConfig = asfWriter as IConfigAsfWriter;

        // Windows Media Video 8 for Dial-up Modem (No audio, 56 Kbps)
        // READ THE README for info about using guids
        Guid cat = new Guid(0x6E2A6955, 0x81DF, 0x4943, 0xBA, 0x50, 0x68, 0xA9,
0x86, 0xA7, 0x08, 0xF6);
        //Guid cat = new Guid(0xAFE69B3A, 0x403F, 0x4A1B, 0x80, 0x07, 0x0E, 0x21,
0xCF, 0xB3, 0xDF, 0x84);
        //AFE69B3A-403F-4A1B-8007-0E21CFB3DF84
        hr = lConfig.ConfigureFilterUsingProfileGuid(cat);
        Marshal.ThrowExceptionForHR( hr );
    }
    finally
    {
        Marshal.ReleaseComObject(pTmpSink);
    }

    return asfWriter;
}

/// <summary> Shut down capture </summary>
private void CloseInterfaces()
{
    int hr;

    try
    {
        if( m_mediaCtrl != null )
        {
            // Stop the graph
            hr = m_mediaCtrl.Stop();
        }
    }
}

```

```
        m_bRunning = false;
    }
}
catch
{
}

#if DEBUG
    // Remove graph from the ROT
    if ( m_rot != null )
    {
        m_rot.Dispose();
        m_rot = null;
    }
#endif

    if (m_FilterGraph != null)
    {
        Marshal.ReleaseComObject(m_FilterGraph);
        m_FilterGraph = null;
    }
}
}
```